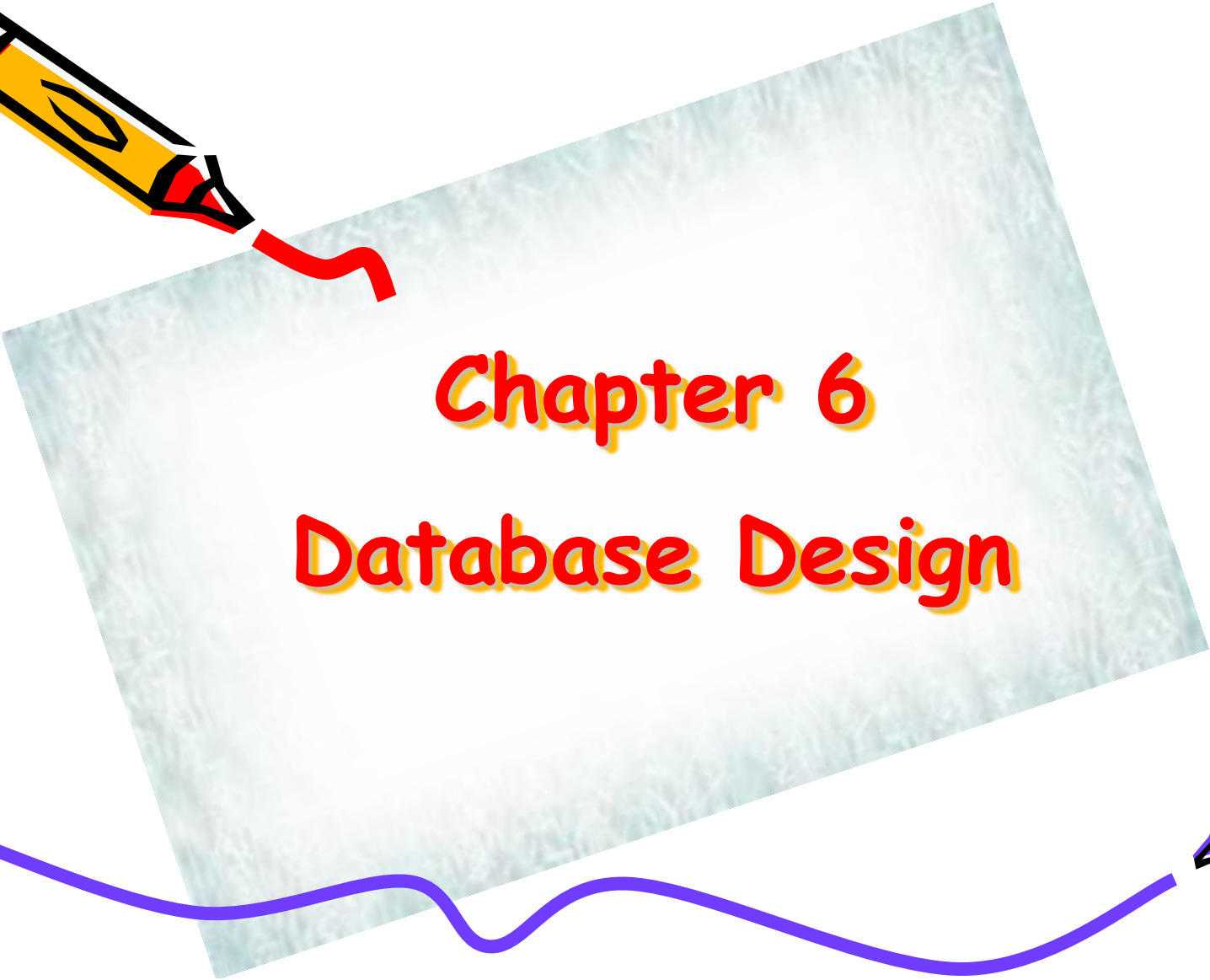
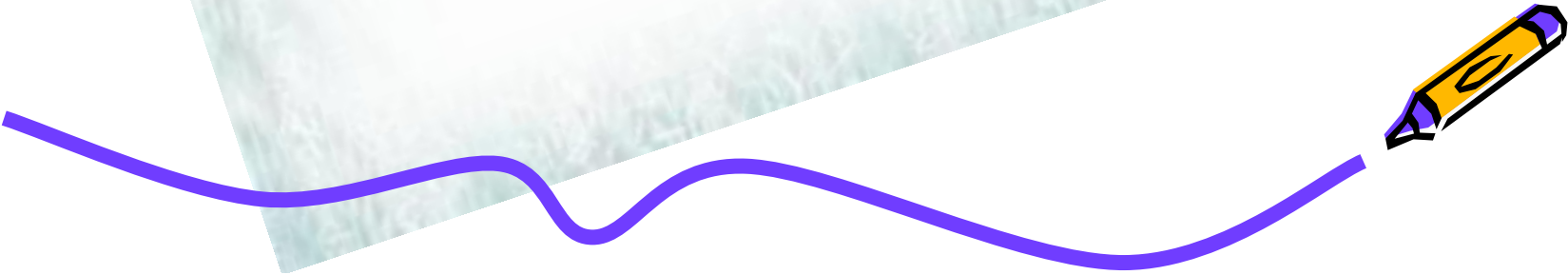




Chapter 6

Database Design



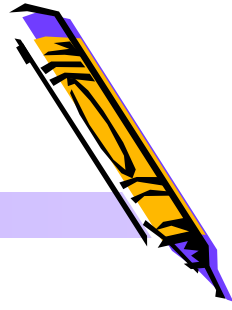
Database Design



School of software Yunnan University

关系数据库是由一组关系组成的，那么针对一个具体问题，应该如何构造一个适合于它的数据模式，即应该构造几个关系，每个关系由哪些属性组成等。这是关系数据库逻辑设计问题。

Database Design

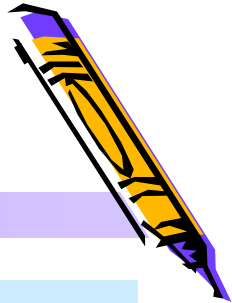


School of software Yunnan University

□ Introduction

- *Logical database design*, also known simply as *database design* or *database modeling*, studies basic properties and interrelationships among data items.
- Two common techniques covered in this chapter are used to performed the task of database design:
 - *entity-relationship* approach (or E-R approach)
 - *normalization* approach
- The most important test of logical design is that the tables and attributes faithfully reflect interrelationships among objects in the real world.

Database Design



School of software Yunnan University

- 6.1 Introduction to E-R Concepts**
- 6.2 Further Details of E-R Modeling**
- 6.3 Additional E-R Concepts**
- 6.4 Case Study**
- 6.5 Normalization: Preliminaries**
- 6.6 Functional Dependencies**
- 6.7 Lossless Decompositions**
- 6.8 Normal Forms**
- 6.9 Additional Design Considerations**



6.1 Introduction to E-R Concepts

School of software Yunnan University

- Three fundamental data classification objects introduced in this section are *entities*, *attributes*, and *relationships*.

□ Entities, Attributes, and Simple E-R Diagrams

- Definition 6.1.1 **Entity**. An entity is a collection of distinguishable real-world objects with common properties.
- CAP database: CUSTOMERS, AGENTS, PRODUCTS, (ORDERS)
- A college database: Students, Instructors, Classrooms, Course ...





6.1 Introduction to E-R Concepts

School of software Yunnan University

□ Entities, Attributes, and Simple E-R Diagrams

- An entity is usually mapped to an actual table, and each row of the table corresponds to one of the distinguishable real-world objects that make up the entity, called **entity instance** (实体实例), or sometimes an **entity occurrence** (实体事件).
- For Example: $E = \{e_1, e_2, \dots, e_n\}$

E或者E1、
E2表示实体

表示实体实例



6.1 Introduction to E-R Concepts

School of software Yunnan University

□ Entities, Attributes, and Simple E-R Diagrams

- Definition 6.1.2 **Attribute**. An attribute is an data item that describes a property of an entity or a relationship.
- A particular *instance* of an entity is said to have **attribute value** for all attributes describing the entity (a null value is possible).





6.1 Introduction to E-R Concepts

School of software Yunnan University

□ Entities, Attributes, and Simple E-R Diagrams

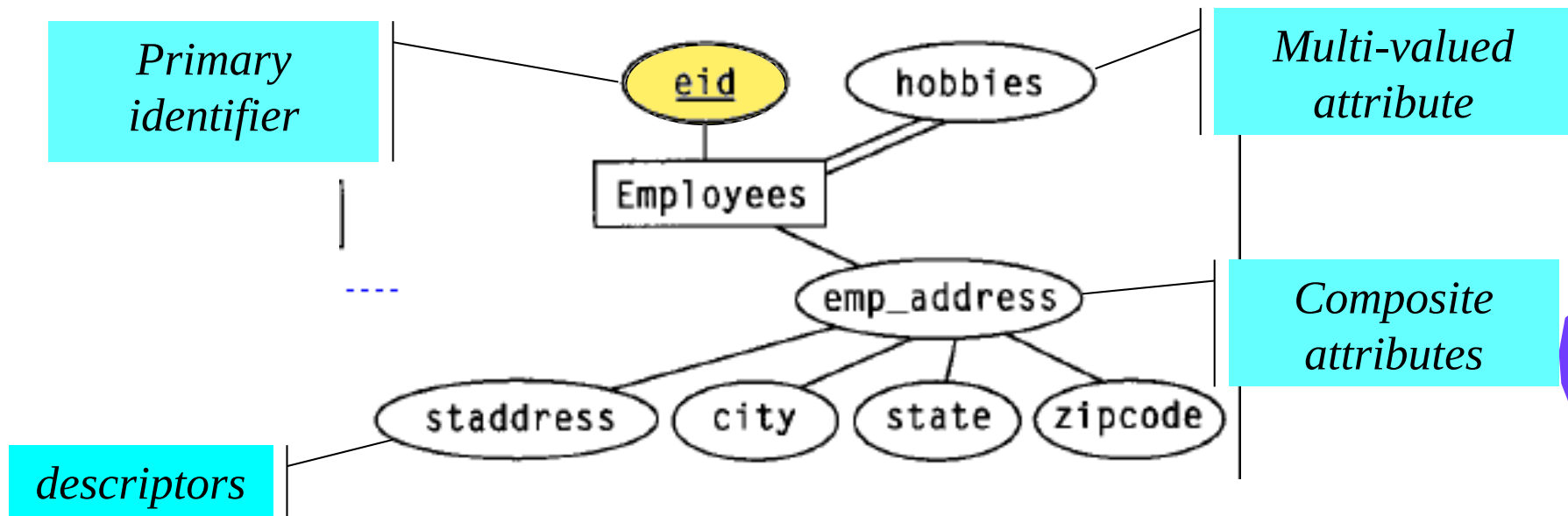
- Each entity has an **identifier** (标志符), an attribute, or set of attributes that taken on unique value for each **entity instance**; this is the analog of the relational concept of **candidate key** (候选键)
 - **primary identifier** (主标志符)
 - **descriptors** (描述符)
 - **composite attribute** (复合属性)
 - **multi-value attribute** (多值属性)



6.1 Introduction to E-R Concepts

School of software Yunnan University

□ Entities, Attributes, and Simple E-R Diagrams





6.1 Introduction to E-R Concepts

School of software Yunnan University

□ Transforming Entities and Attributes to Relations

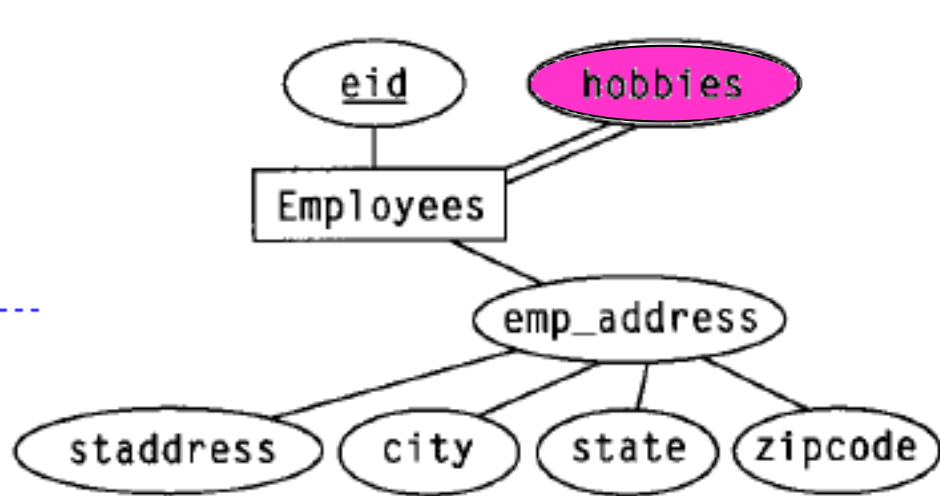
- **转换规则1** E - R图中的每一个**实体**映射到关系数据库中的一个表，并用实体名来命名这个表。表的列代表了连接到实体的所有简单单值属性（可能是通过复合属性连接到实体的，但复合属性本身并不变成表的列）。实体的标识符映射为该表的候选键，实体的**主标识符**映射为主键。注意到实体的主标识符可以是一个复合属性，所以它将变成为关系表中的一个属性集合。**实体实例**映射为该表中的行。

6.1 Introduction to E-R Concepts



School of software Yunnan University

□ Transforming Entities and Attributes to Relations



Transformation Rule 1

employees

<u>eid</u>	staddress	city	state	zipcode
197	7 Beacon St	Boston	MA	02122
...	...	...	...	...



6.1 Introduction to E-R Concepts

School of software Yunnan University

□ Transforming Entities and Attributes to Relations

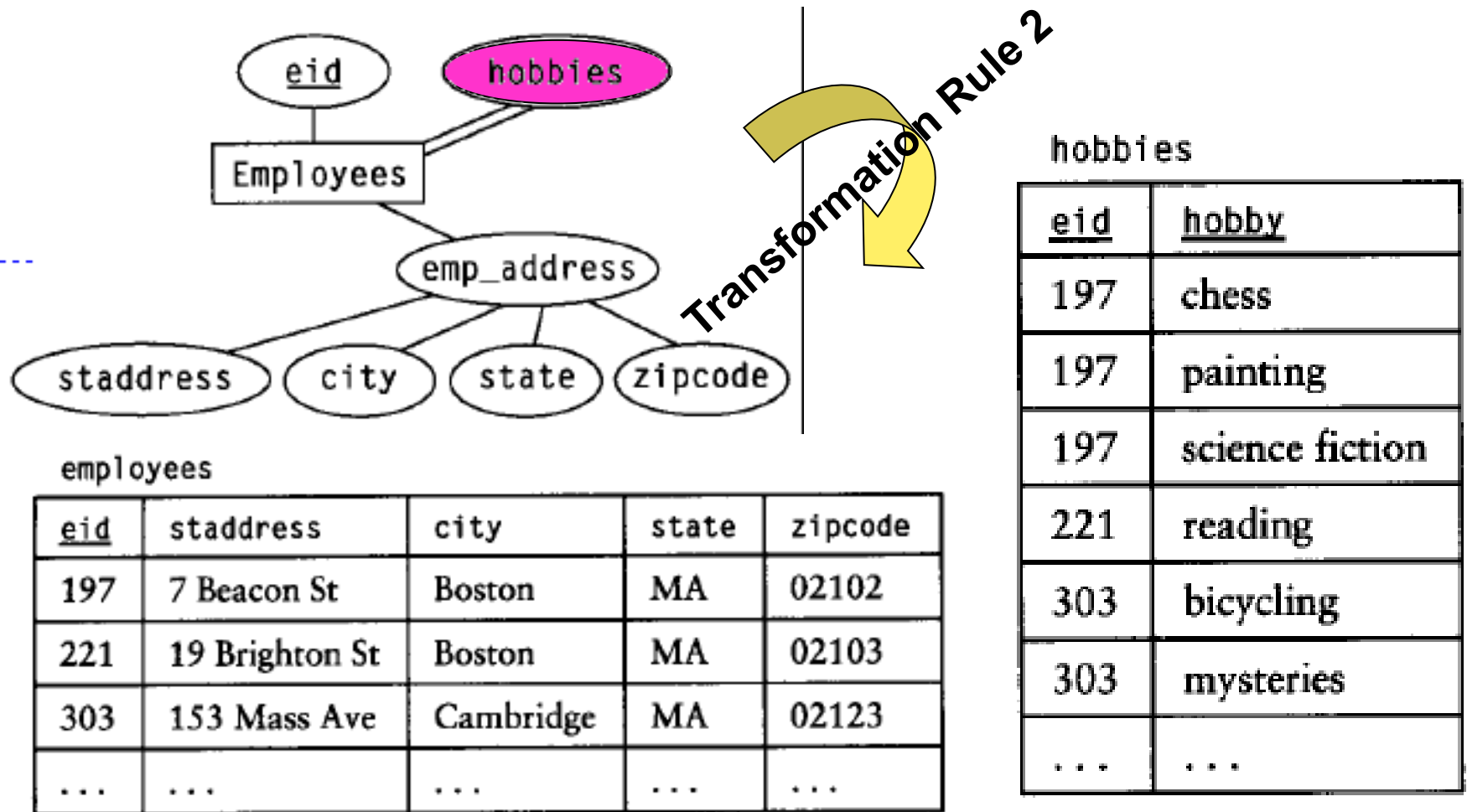
- **转换规则2** 给定一个实体 E ，主标识是 p 。一个多值属性 a 在 $E - R$ 图中连接到 E ，那么 a 映射成自身的一个表，该表按照复数形式的多值属性名命名。这个新表的列用 p 和 a 命名（ p 或 a 都可能由几个属性组成），表的行对应（ p, a ）值对，表示与 E 的实体实例关联的 a 的属性值对。这个表的主键属性是 p 和 a 中列的集合。



6.1 Introduction to E-R Concepts

School of software Yunnan University

□ Transforming Entities and Attributes to Relations



6.1 Introduction to E-R Concepts



School of software Yunnan University

□ Relationship among Entities

- **Definition 6.1.3 Relationship.** 给定 m 个实体的有序列表： E_1 、 E_2 ，...， E_m （列表中同一个实体可以出现多于一次），一个联系 R 定义了这些实体实例之间的对应规则。特别地， R 代表了一个 m 元组的集合，它是笛卡尔积 $E_1 \times E_2 \times \dots \times E_m$ 的子集。
- A particular occurrence of a relationship, is called a **relationship occurrence**（联系事件） or **relationship instance**（联系实例）.
- The number of entities m in the defining list is called the **degree**（度） of the relationship.

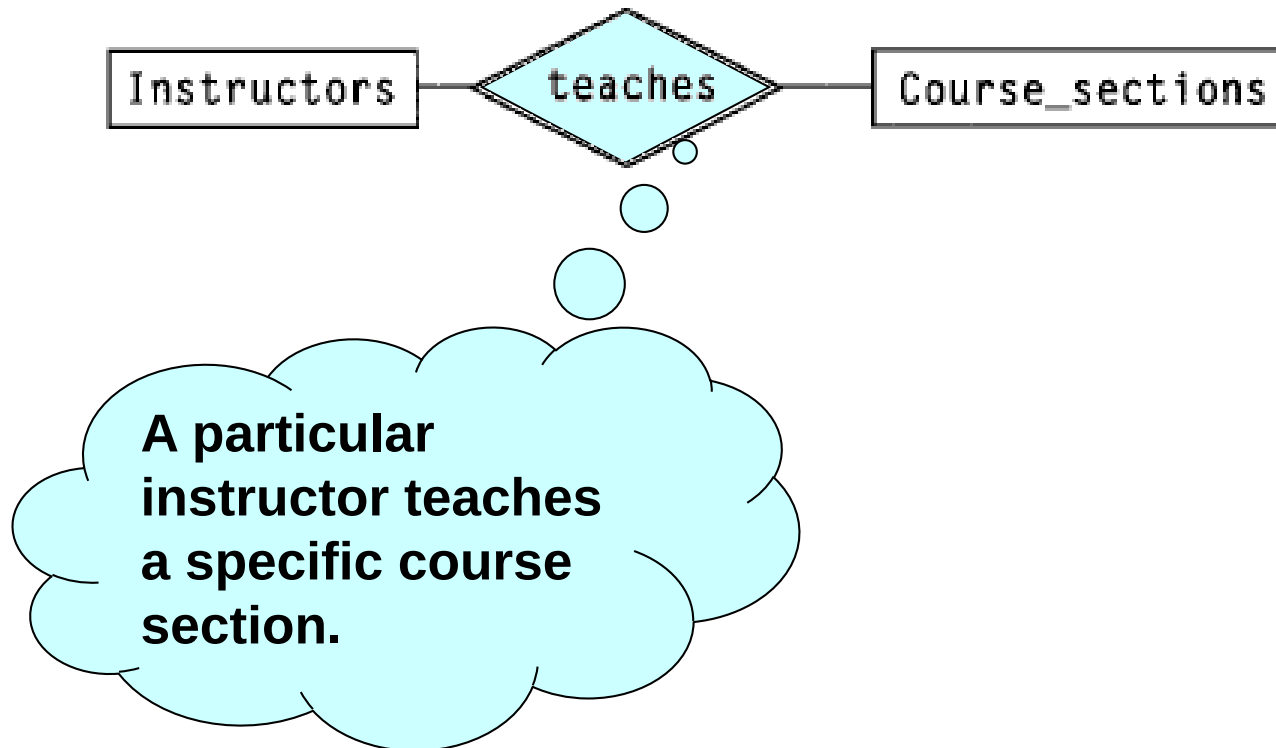


6.1 Introduction to E-R Concepts

School of software Yunnan University

□ Relationship among Entities

- A relationship between two entities is known as a *binary relationship* (二元联系).



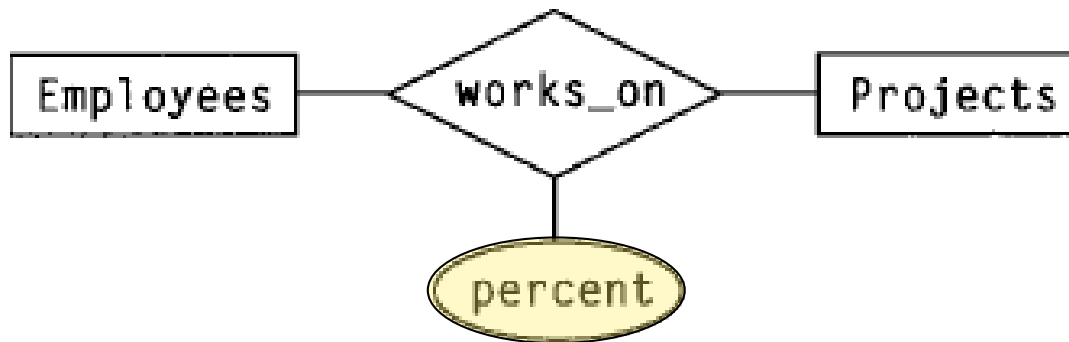


6.1 Introduction to E-R Concepts

School of software Yunnan University

□ Relationship among Entities

- A relationship can also have *attached attributes* (附加属性).
- For Example: The relationship *Works_on* might have the attribute *percent*.



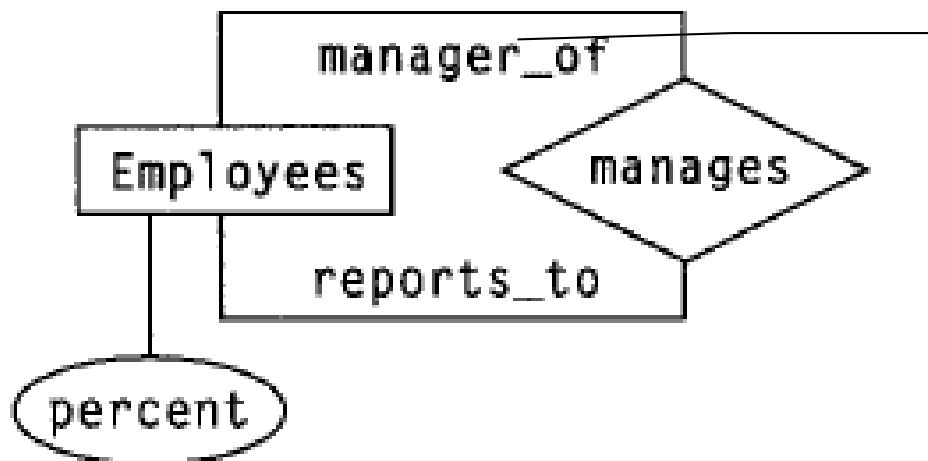


6.1 Introduction to E-R Concepts

School of software Yunnan University

□ Relationship among Entities

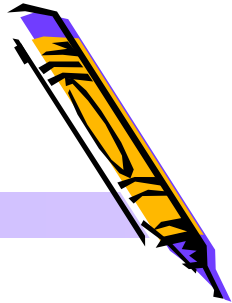
- A binary relationship that relates an entity to itself is called a **ring** (环), or sometimes a **recursive relationship** (递归联系).



In the case of a ring, the connecting lines are often labeled with the name of the roles played by the entity instances involved.

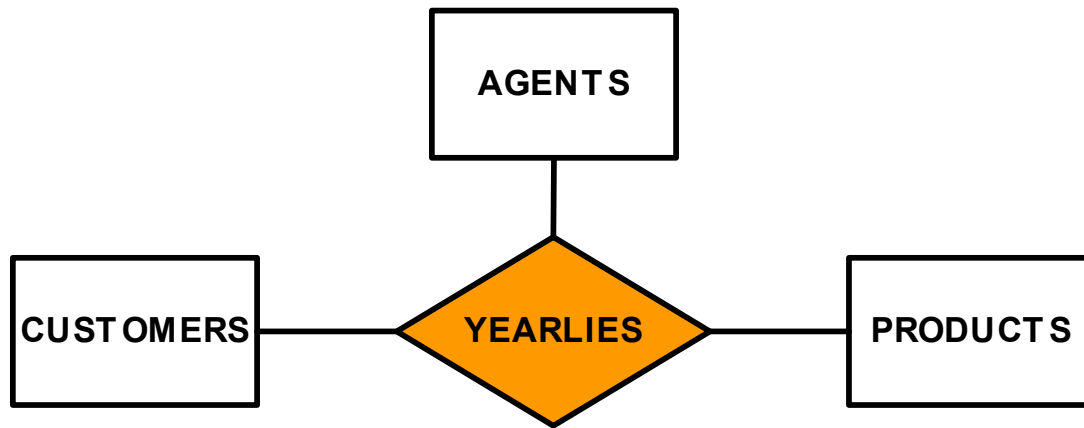
6.1 Introduction to E-R Concepts

School of software Yunnan University



□ Relationship among Entities

■ Ternary relationship



6.3 Additional E-R Concepts



School of software Yunnan University

□ Relationship among Entities

- 例6.1.3 CAP中的表 **orders**并不代表一个联系根据定义 6.1.3，CAP数据库中的表**orders**不是**Customers**、**Agents**和**Products**之间的联系。这是因为表 **orders**行中的三元组（ **cid** ， **aid**， **pid** ）不能像要求的那样确定笛卡儿积 **Customers**×**Agents** ×**Products**的一个子集。相反的，在图 2 - 2的例子中，三元组（ **cid**， **aid**， **pid**）的一些值出现不止一次，这显然是设计者的意图，因为同一个顾客可以两次从同一个代理商处订购同一种产品。

6.3 Additional E-R Concepts



School of software Yunnan University

□ Relationship among Entities

- 例6.1.4 假设我们要进行一项调查，在调查中需要知道当年所有交易的总额，而这要从`customers`，`agents`和`products`生成的`orders`表中得到。例如，我们可能要调查`agents`和`customers`之间的交易量关系，也可能调查`customers`和`products`之间的交易量关系，以及那些关系是如何被地理因素（`city`的值）影响的。然而，开始设计时，我们发现不断地由表`orders`计算总量以获得这个基本数据十分低效，所以我们决定建一个新表`yearlies`。用下面的SQL命令定义这个新表：

6.3 Additional E-R Concepts



School of software Yunnan University

□ Relationship among Entities

```
create table yearlies (cid char(4), aid char(3), pid char(3),  
    totqty integer, totdoll float);  
insert into yearlies  
    select cid, aid, pid, sum(qty), sum(dollars) from orders  
    group by cid, aid, pid;
```

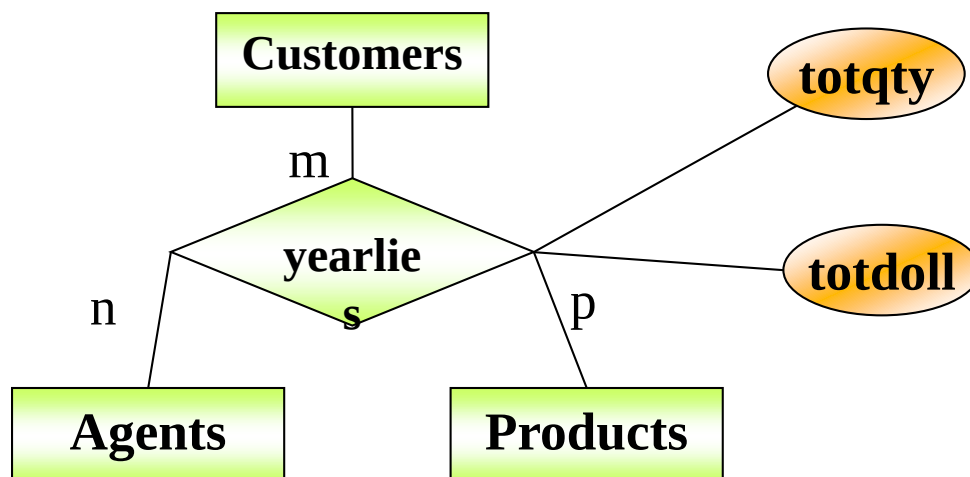
6.3 Additional E-R Concepts



School of software Yunnan University

□ Relationship among Entities

- `yearlies`表是一个联系，因为表中行的三元组 (`c id, a id, p id`) 可以确定笛卡尔积 `Customers × Agents × Products` 的一个子集。





6.1 Introduction to E-R Concepts

School of software Yunnan University

概念	描述	例子
实体	现实世界中具有共同性质的可识别对象的集合	Customers, Agents, Products, Employees
属性	描述实体或者联系的某个性质的数据项	见下面
标识符 (属性的集合)	唯一地标识一个实体或者联系实例	顾客标识符 cid , 雇员标识符 eid
描述符	非键属性, 描述一个实体或者联系	city (Customers中), capacity (Class_rooms中)
复合属性	共同描述一个对象的某个性质的一组简单属性	emp_address (参见图6-2)
多值属性	对于一个实体实例可以取多个值的实体属性	hobbies (参见图6-2)

图6-5a 基本E-R概念：实体和属性



6.1 Introduction to E-R Concepts

School of software Yunnan University

概 念	描 述	例 子
关系	命名的m元组集合, 标识笛卡儿积 $E_1 \times E_2 \times \dots \times E_m$ 的子集	
二元关系	两个不同实体上的联系	teaches, works_on (参见图6-3)
环, 递归关系	联系一个实体到自身的联系	manages (参见图6-3)
三元关系	三个不同实体上的联系	yearlies (参见例6.1.4)

图6-5b 基本E-R概念: 联系



6.2 Further Detail of E-R Modeling



School of software Yunnan University

□ Cardinality of Entity Participation in a Relationship

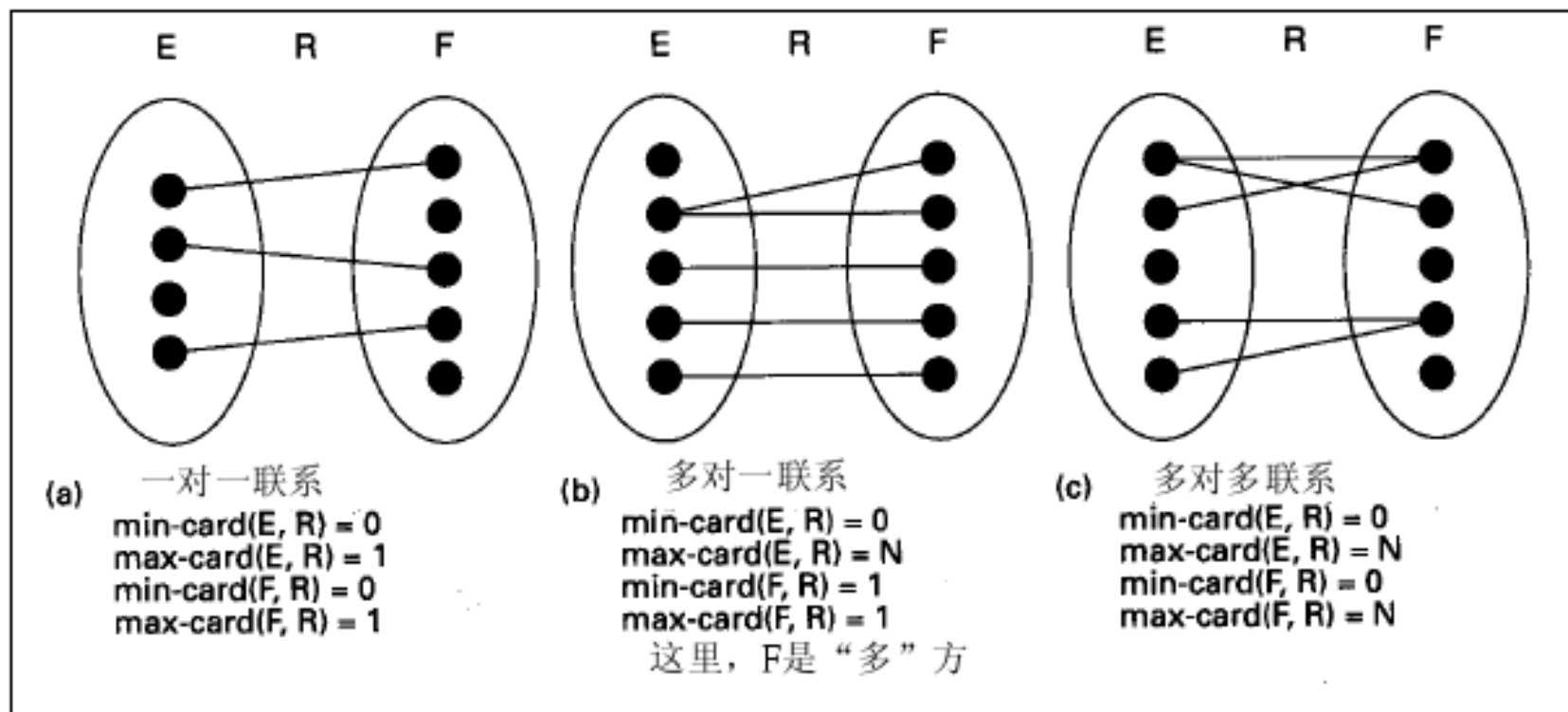


图6-6 两个实体E和F之间联系R的例子

6.2 Further Detail of E-R Modeling



School of software Yunnan University

□ Cardinality of Entity Participation in a Relationship

- Definition 6.2.1 一个实体E参与联系R，并且 $\text{min-card}(E, R) = x$ (x为0或1)， $\text{max-card}(E, R) = y$ (y为1或N)，那么在E-R图中，E和R之间的连接线可以用有序对 (x, y) 来标记。我们使用一个新的标记来表示这个最小最大有序对 (x, y) ：

$\text{card}(E, R) = (x, y)$ 。

6.2 Further Detail of E-R Modeling



School of software Yunnan University

□ Cardinality of Entity Participation in a Relationship

- Example 6.2.1 consider the value of min-card() and max-card()



- min-card(Instructors, teaches)=0
- max-card(Instructors, teaches)=N

card(Instructors,teaches)=(0,N)

- min-card(Course_sections, teaches)=1
- max-card(Course_sections, teaches)= 1

6.2 Further Detail of E-R Modeling



School of software Yunnan University

Cardinality of Entity Participation in a Relationship

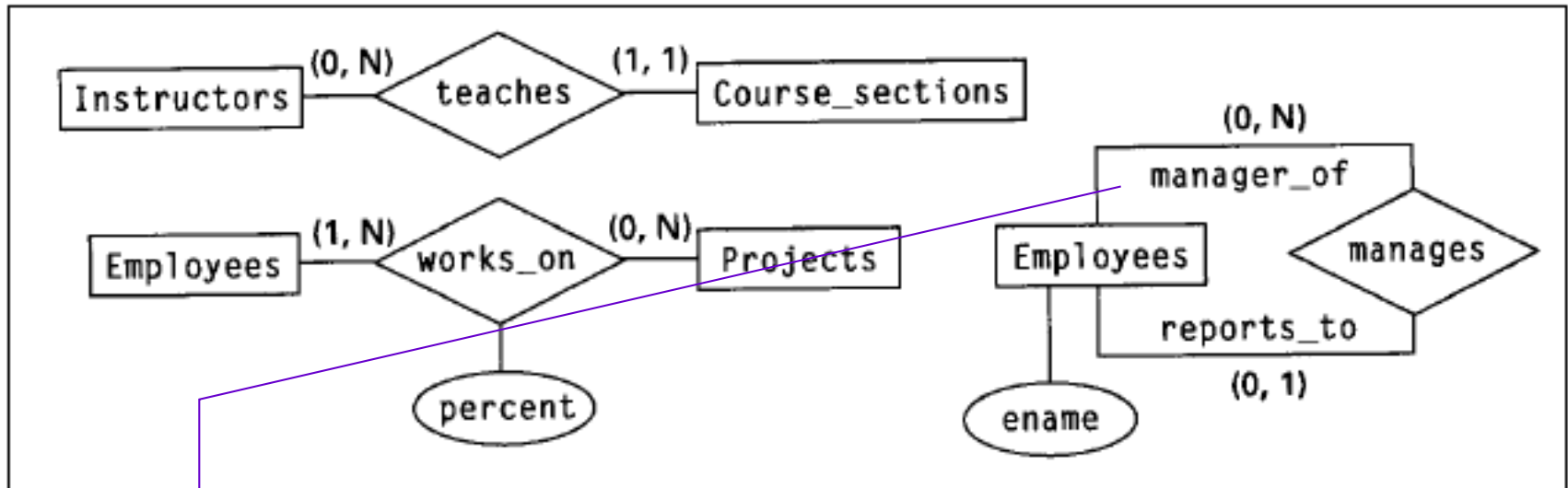


图6-7 E-R图：实体联系间的连接用(x,y)标示

we use the cardinality notation with entities having parenthesized roles to remove ambiguity:
 $\text{card}(\text{Employees}(\text{reports_to}), \text{manages}) = (0, 1)$
 $\text{card}(\text{Employees}(\text{manager_of}), \text{manages}) = (0, N)$

6.2 Further Detail of E-R Modeling

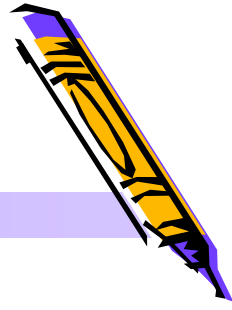


School of software Yunnan University

□ One-to-one, Many-to-Many, and Many-to-one Relationships

- Definition 6.2.2 当一个实体 E 在联系 R 中具有 $\max\text{-card}(E, R) = 1$ 时, E 被称为在联系 R 中是单值的。如果 $\max\text{-card}(E, R) = N$, 那么 E 称为在联系中是多值的。如果实体 E 和 F 在联系中都是多值的, 则 E 和 F 之间的二元联系 R 称为是多对多的或 $N - N$ 的。如果 E 和 F 都是单值的, 那么这个联系称为是一对一的或 $1 - 1$ 的。如果 E 是单值的而 F 是多值的或者相反, 那么这个关系称为是多对一的或 $N - 1$ 的 (我们通常不区别 $1 - N$ 联系和 $N - 1$ 联系)。

6.2 Further Detail of E-R Modeling



School of software Yunnan University

□ One-to-one, Many-to-Many, and Many-to-one Relationships

- The values $\text{max-card}(E,R)$ and $\text{max-card}(F,R)$ determine whether a binary relationship is many-to-many, many-to-one, or one-to-one.
- The values $\text{min-card}(E,R)$ and $\text{min-card}(F,R)$ are not mentioned, and they are said to be independent of these characterizations.



6.2 Further Detail of E-R Modeling



School of software Yunnan University

□ One-to-one, Many-to-Many, and Many-to-one Relationships

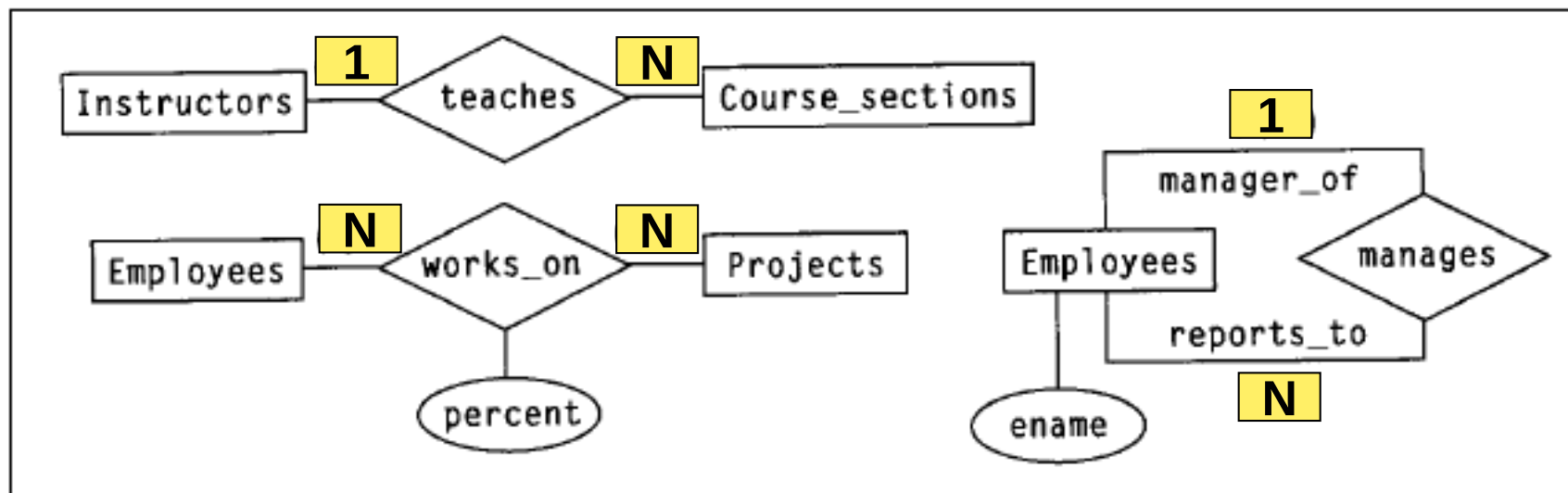


图6-7 E-R图：实体联系间的连接用(x,y)标示

6.2 Further Detail of E-R Modeling



School of software Yunnan University

□ Transforming Binary Relationships to Relations

- Transformation Rule 3. N-N Relationships. 当两个实体 **E** 和 **F** 参与一个 **多对多** 二元联系 **R** 时，在相关的关系数据库设计中，**联系映射成一个表 T**。这个表包括从实体 **E** 和 **F** 转化而来的两个表的主键的所有属性，这些列构成了表 **T** 的主键。**T** 还包含了连结到联系的所有属性的列。联系实例用表的行表示，相关联的实体实例可以通过这些行的主键值唯一地标识出。

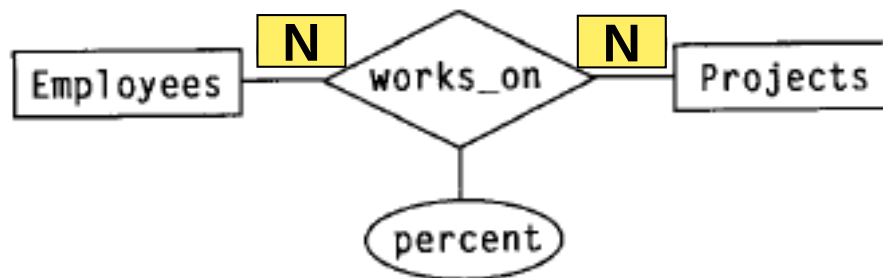
6.2 Further Detail of E-R Modeling



School of software Yunnan University

□ Transforming Binary Relationships to Relations

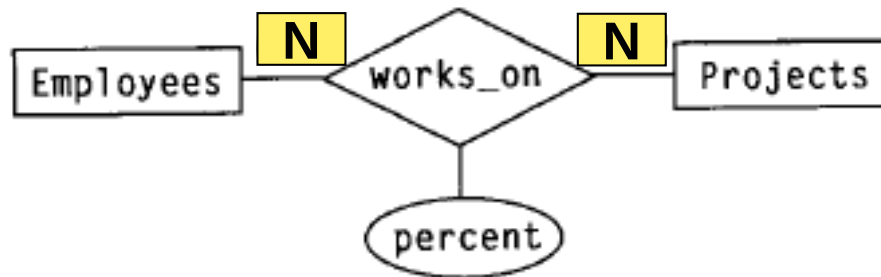
■ Example 6.2.2 N-N Relationships



6.2 Further Detail of E-R Modeling



School of sof



employees

<u>eid</u>	straddr	city	state	zipcode
197	7 Beacon St	Boston	MA	02102
221	19 Brighton St	Boston	MA	02103
303	153 Mass Ave	Cambridge	MA	02123
...	...	...	...	...

projects

<u>prid</u>	proj_name	due_date
p11	Phoenix	3/31/99
p13	Excelsior	9/31/99
p21	White Mouse	6/30/00
...	...	...

works_on

<u>eid</u>	<u>prid</u>	percent
197	p11	50
197	p13	25
197	p21	25
221	p21	100
303	p13	40
303	p21	60
...	...	...

6.2 Further Detail of E-R Modeling



School of software Yunnan University

□ Transforming Binary Relationships to Relations

- Transformation rule 4. N-1 Relationships. 当两个实体 E 和 F 参与一个多对一的二元联系 R 时，联系在关系数据库设计中不能映射成一个表。相反，如果我们假设实体 F 具有 $\max\text{-card}(F, R) = 1$ ，并表示联系中的“多”方，那么从实体 F 转化成的关系表 T 中应当包括从实体 E 转换出的关系表的主键属性列，这被称为 T 的外键。因为 $\max\text{-card}(F, R) = 1$ ， T 的每一行都通过一个外键值联系到实体 E 的一个实例。如果 F 在 R 中是强制参与的，那么它必须恰恰与 E 的一个实例相联系，这意味着 T 的上述外键不能取空值。如果 F 在 R 中是选择参与的，那么 T 中不与 E 的实例相联系的行在外键所有列可以取空值。

6.2 Further Detail of E-R Modeling



School of software Yunnan University

□ One-to-one, Many-to-Many, and Many-to-one Relationships

■ Definition 6.2.3

当一个联系 R 中的实体 E 具有 $\text{min-card}(E, R)=1$ 时, E 称为**强制参与** (**mandatory participation**) R , 或者简单地称为在 R 中是强制的。一个实体 E 在 R 中不是强制的 $\text{min-card}(E, R)=0$, 则称为可选的, 或者称为具有**选择参与** (**optional participation**)。

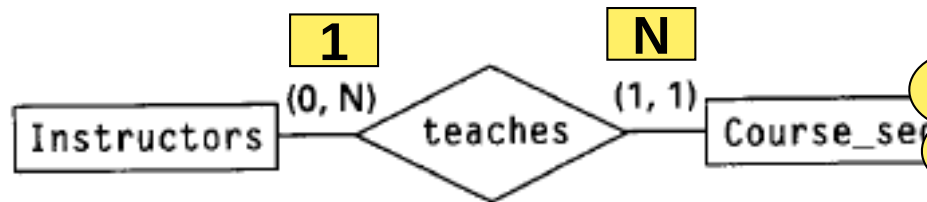
6.2 Further Detail of E-R Modeling



School of software Yunnan University

Transforming Binary Relationships to Relations

Example 6.2.3 N-1 Relationships



**referential
integrity**

instructors				course_sections				
<u>insid</u>	lname	office_no	ext	<u>secid</u>	insid	course	room	period
309	O'Neil	S-3-223	78543	120	309	CS240	M-1-213	MW6
123	Bergen	S-3-547	78413	940	309	CS630	M-1-214	MW7:30
113	Smith	S-3-115	78455	453	123	CS632	M-2-614	TTH6
...	...	...	...	...	...	...	...	...

foreign key

**强制参与，所以insid
NOT NULL**

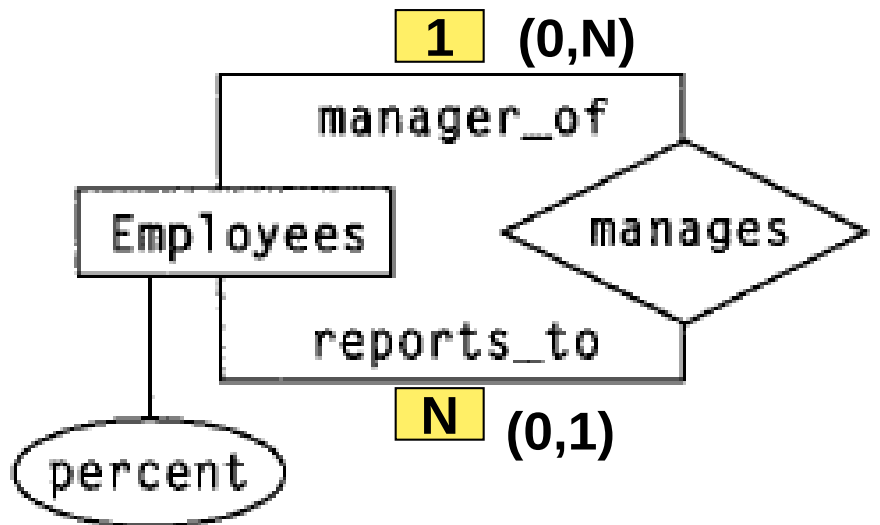
6.3 Additional E-R Concepts



School of software Yunnan University

Relationship among Entities

- In converting an E-R design to a relational one, a relationship is sometimes translated into a relational table, and sometimes not.



employees

eid	ename	mgrid
e001	Jacqueline	null
e002	Frances	e001
e003	Jose	e001
e004	Deborah	e001
e005	Craig	e002
e006	Mark	e002
e007	Suzanne	e003
e008	Frank	e003
e009	Victor	e004
e010	Chumley	e007

6.2 Further Detail of E-R Modeling



School of software Yunnan University

□ Transforming Binary Relationships to Relations

- Transformation rule 5. 1-1 Relationships, Optional participation. 给定两个实体 **E** 和 **F**，他们参与一对一二元联系 **R**，二者的参与都是可选的。我们希望将这一情形转换为关系设计。为此，我们首先按照转换规则1建立表 **S** 来表示实体 **E**；同样建立表 **T** 表示实体 **F**。然后我们向表 **T** 中添加一组列（作为外键），这些列在表 **S** 构成主键。对于 **R** 的任何联系实例，外键列填写的值引用另一张表相应行，这一联系是 **R** 的实例确定的。

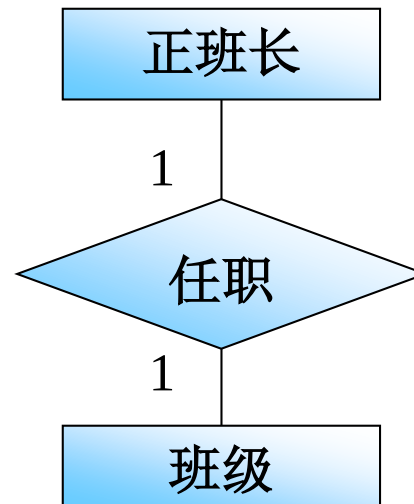
6.2 Further Detail of E-R Modeling



School of software Yunnan University

□ Transforming Binary Relationships to Relations

- Transformation rule 6. 1-1 Relationships, Mandatory Participation on Both sides. 对于一个两方都是强制参与的一对一联系，最好将两个实体对应的两个表合并成为一个表，这样可以避免使用外键。



6.3 Additional E-R Concepts



School of software Yunnan University

□ Relationship among Entities

- A relationship on more than two entities is called an *N-ary relationship* (N元联系).
- Usually such an N-ary relationship is transformed into a table of its own.
- But if all but one of the entities of the relationship participate with max-card=1, then it is possible to represent the relationship with N-1 foreign keys in the one table that participates with greater cardinality.

6.3 Additional E-R Concepts



School of software Yunnan University

□ Cardinality of Attributes 属性的基数

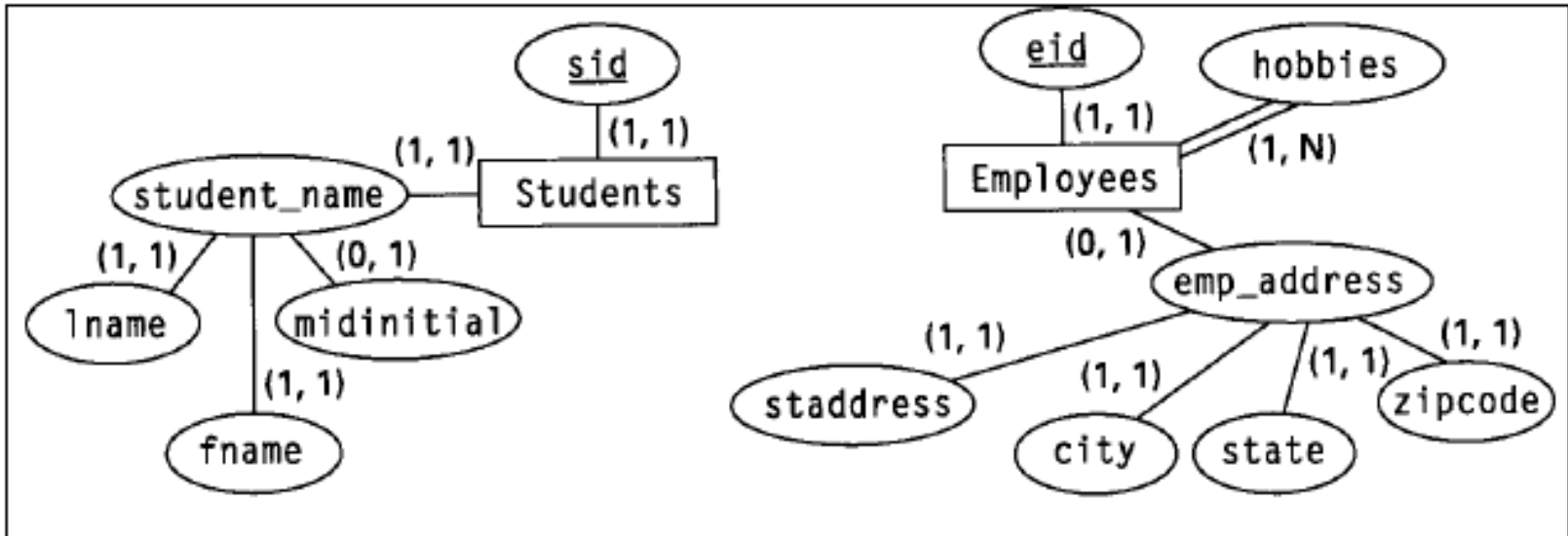
- **Definition 6.3.1** 给定一个实体 E 和隶属于它的属性 A ，我们用 $\text{min-card}(A, E) = 0$ 来表示属性 A 是可选的，用 $\text{min-card}(A, E) = 1$ 来表示属性 A 是强制的。一个强制的属性应当与一个列相对应，这个列在实体 E 对应的表中声明且不能取空值。我们用 $\text{max-card}(A, E) = 1$ 来表示属性是单值的， $\text{max-card}(A, E) = N$ 来表示属性是多值的。一个属性 A ，当 $\text{min-card}(A, E) = x$ 且 $\text{max-card}(A, E) = y$ 时，就写为 $\text{card}(A, E) = (x, y)$ 。 (x, y) 可以用来标记 $E - R$ 图中属性 - 实体间的连接，以显示这个属性的基数。

6.3 Additional E-R Concepts



School of software Yunnan University

□ Cardinality of Attributes 属性的基数



6.3 Additional E-R Concepts



School of software Yunnan University

□ Weak Entities

- Definition 6.3.2 如果实体的所有实例都通过一个联系 R 依赖于另一个实体的实例而存在，那么这个实体就称为**弱实体**，而另一个实体称为**强实体**。

6.3 Additional E-R Concepts



School of software Yunnan University

Weak Entities

Example:

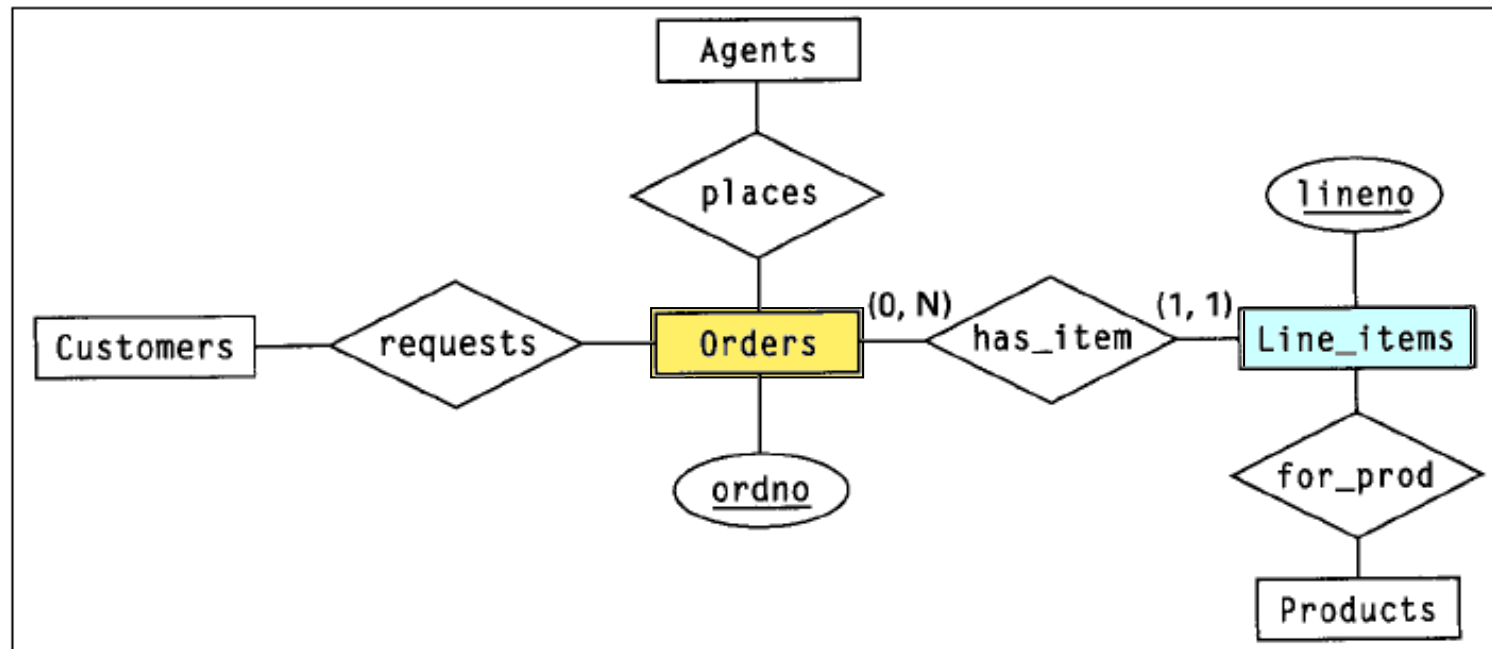
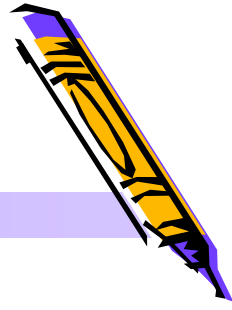


Figure 6.11 A weak Entity, **Line_items**, Dependent on the Entity **Orders**

6.3 Additional E-R Concepts



School of software Yunnan University

□ Weak Entities

- If the Orders occurrence goes away, all occurrences of the weak entity Line_items will likewise disappear.
- The primary identifier for the weak entity Line_items must include the attributes in the primary identifier for the orders entity. Thus the primary key for the line_items table is constructed from the external attributes **ordno** and the weak entity identifier **lineno**.



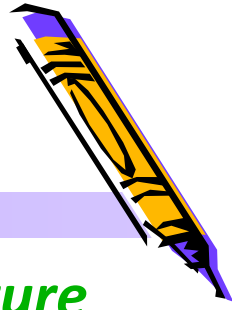
6.4 Case Study



School of software Yunnan University

- Consider a **simple airline reservation database** handling outgoing flights from one airline terminal.
- We need to keep track of **passengers**, **flights**, **departure gates**, and **seat assignments**
- **Passengers** — with primary identifier attribute **ticketno**
- **Flights** — having primary identifier **flightno** (unique identifier values, not repeated on successive days) and descriptive attribute **depart_time** (actually made up of **date** and **time**)

6.4 Case Study



School of software Yunnan University

- We need to keep track of *passengers*, *flights*, *departure gates*, and *seat assignments*
- **Gates**—with primary identifier *gateno*
- **Seats**—identified by a seat number, *seatno*, valid only for a specific flight.
- The *seatno* primary identifier for Seats is only meaningful when related to a Flights instance.
- Seats is *weak entity*, and thus there must be a relationship between Flights and Seats, which we name *has_seat*. The identifier for Seats containing the identifier of Flights.

6.4 Case Study



School of software Yunnan University

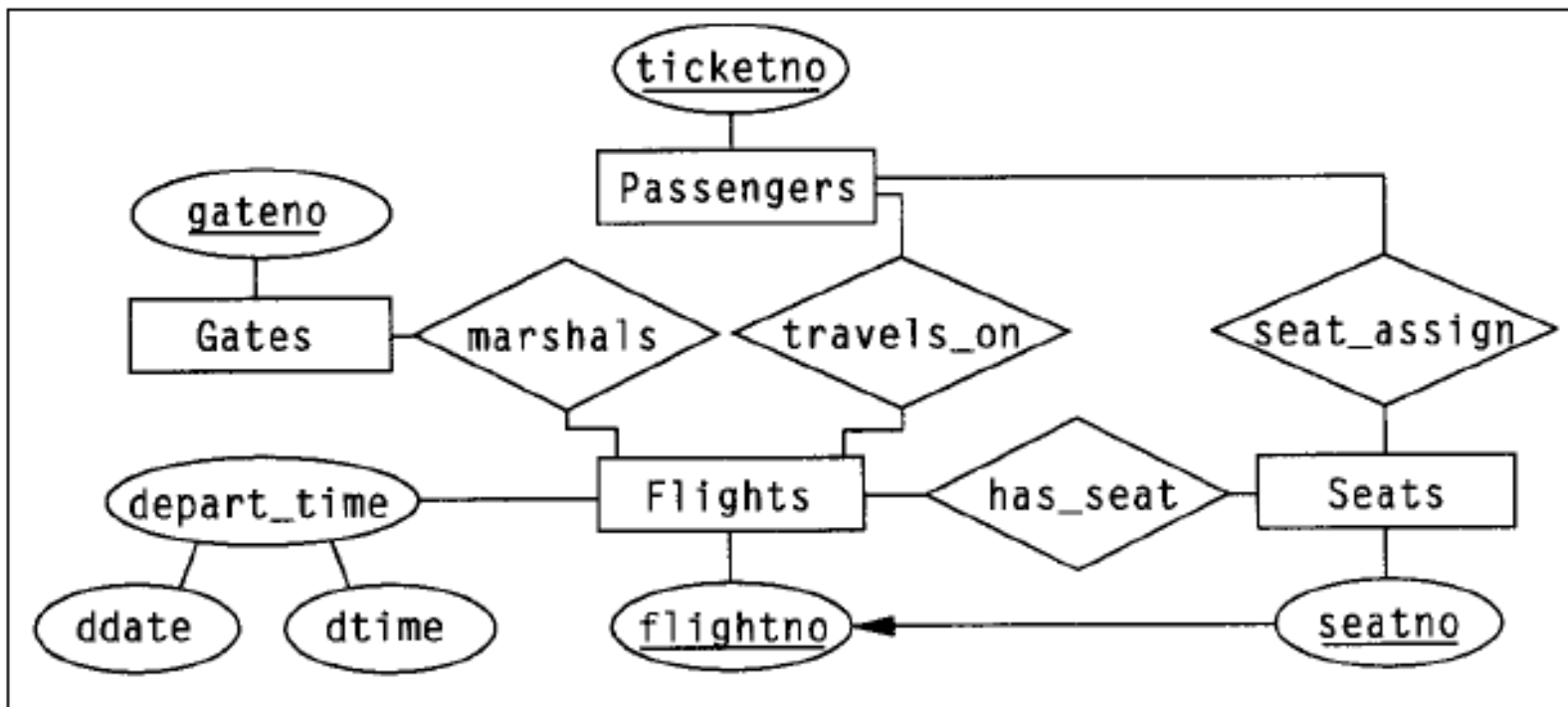
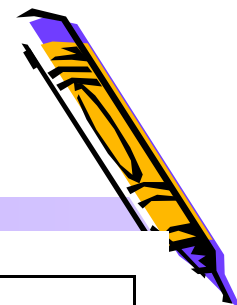


图6-13 简单航空订票系统数据库的早期E-R设计

6.4 Case Study



School of software Yunnan University

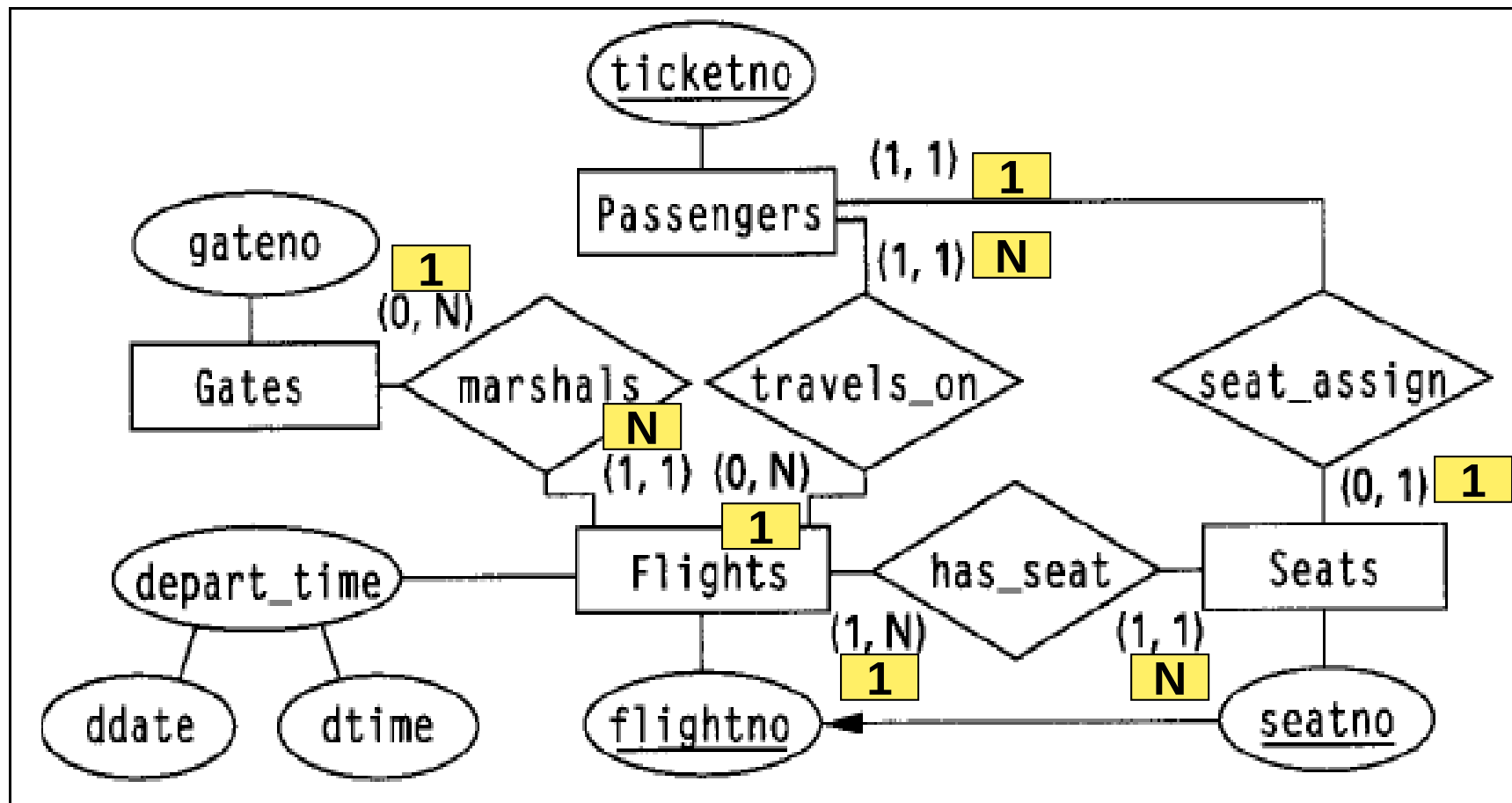
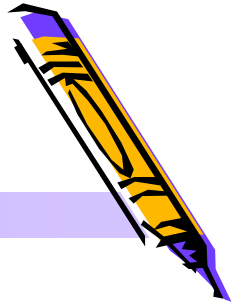


图6-14 带有基数的简单航空订票数据库的E-R图

6.4 Case Study



School of software Yunnan University

- Create tables to map entities

passengers

<u>ticketno</u>
. . .

gates

<u>gateno</u>

flights

<u>flightno</u>	ddate	dtime

seats

<u>seatno</u>

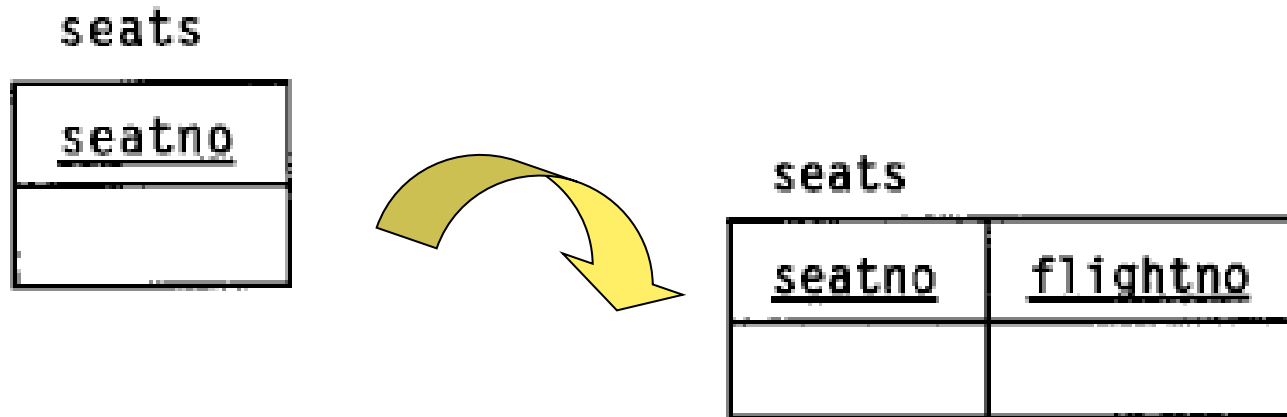


6.4 Case Study



School of software Yunnan University

- Consider the relationship **has_seat**, which is **N-1**, with Seats on “many” side.
- By Transformation RULE 4, a **foreign key** in the seats table will connect each seats row to the appropriate flights row.
- Seats is **weak entity**...

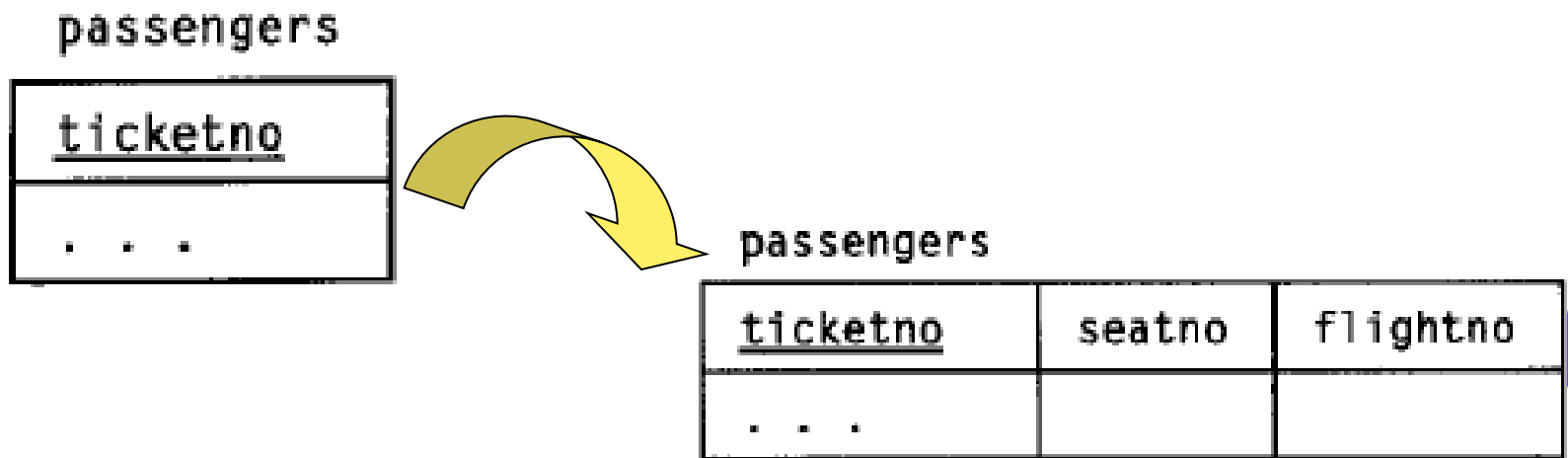


6.4 Case Study



School of software Yunnan University

- The **seat_assign** relationship is **1-1**, with optional participation by Seats, so by Transformation RULE 5 we can represent this by adjoining to the passengers table a **foreign key** for seats.



6.4 Case Study

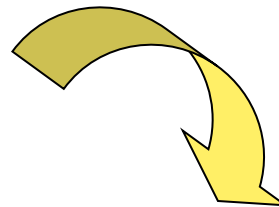


School of software Yunnan University

- Consider the **marshals** relationship. This is **N-1**, with **Flights** on the “many” side, so by Transformation RULE 4 a foreign key in the flights table, gateno, will connect each flights row to the appropriate gate now.

flights

<u>flightno</u>	ddate	dtime



flights

<u>flightno</u>	gateno	ddate	dtime

6.4 Case Study



School of software Yunnan University

- The **travels_on** relationship is **N-1**, with Passengers on the “many” side. A foreign key flightno in the passengers table will connect each passengers row to the appropriate Flights row.

passengers

<u>ticketno</u>	seatno	flightno
. . .		

gates

<u>gateno</u>

flights

<u>flightno</u>	gateno	ddate	dtime

seats

<u>seatno</u>	<u>flightno</u>

6.5 Normalization: Preliminaries



School of software Yunnan University

- In the normalization approach, the designer starts with a real-world situation to be modeled and lists the **data items** that are candidates to become column names in relational tables, together with **a list of rules** about the relation of these data items.
- Normal forms
 - First/Second normal form(1NF/2NF)
 - Third normal form(3NF)
 - Boyce-Codd normal form(BCNF)
 - 4NF,5NF

6.8 Normal Forms



School of software Yunnan University

□ First Normal Form (1NF)

- **Definition:** A table in which all fields contain single values only.
- **定义:** 如果一个关系模式 R 的所有属性都是不可分的基本数据项, 则 $R \in 1NF$ 。
- 在任何一个关系数据库中, 第一范式是对关系模式的一个最起码的要求。不满足第一范式的数据库模式不能称为关系数据库。但是, 满足第一范式的关系模式不一定是一个好的关系模式。

6.5 Normalization: Preliminaries



School of software Yunnan University

□ A Running Example: Employee Information

emp_id	
emp_name	
emp_phone	
dept_name	
dept_phone	
dept_mgrname	
skill_id	} 从一个到大量的技能可用于一个公司
skill_name	
skill_date	
skill_lvl	

图6-15 没有规范化的雇员信息数据项

6.5 Normalization: Preliminaries



School of software Yunnan University

□ A Running Example: Employee Information

emp_info

emp_id	emp_name	...	skill_id	skill_name	skill_date	skill_lvl
09112	Jones	...	44	librarian	03-15-99	12
09112	Jones	...	26	PC-admin	06-30-98	10
09112	Jones	...	89	word-proc	01-15-00	12
12231	Smith	...	26	PC-admin	04-15-99	5
12231	Smith	...	39	bookkeeping	07-30-97	7
13597	Brown	...	27	statistics	09-15-99	6
14131	Blake	...	26	PC-admin	05-30-98	9
14131	Blake	...	89	word-proc	09-30-99	10
...	...	...	...	...	...	...

图6-16 单个雇员信息表 emp_info，满足第一范式

6.5 Normalization: Preliminaries



School of software Yunnan University

□ Anomalies of a Bad Database Design

- Definition 6.5.1 **Update Anomaly**(更新异常). A table T is subject to an update anomaly when changing a single attribute value for an entity instance or relationship instance represented in the table may require that several rows of T be updated.
- 如果更改表所对应的某个实体实例或关系实例的单个属性时，需要更新多行，那么就说这个表存在更新异常（更新复杂）。

6.5 Normalization: Preliminaries



School of software Yunnan University

□ Anomalies of a Bad Database Design

- Definition 6.5.2 **Delete Anomaly, Insert Anomaly**. 如果删除表的某一行来反映某个实体实例或者关系实例消失时，会导致丢失另一个不同实体实例或者关系实例的信息，而这是我们不希望丢失的，那么就说这个表存在删除异常。插入异常是这个问题在插入方面的表现，即我们无法在缺少另一个实体实例或关系实例的情况下，表示某个实体或者实例的信息。

6.5 Normalization: Preliminaries



School of software Yunnan University

□ Anomalies of a Bad Database Design

- To factor tables so that all **anomalies are removed** from our representation.

emps	skills
emp_id	emp_id
emp_name	skill_id
emp_phone	skill_name
dept_name	skill_date
dept_phone	skill_lvl
dept_mgrname	

图6-17 具有两个表的emp_info 数据库

6.5 Normalization: Preliminaries



School of software Yunnan University

□ Anomalies of a Bad Database Design

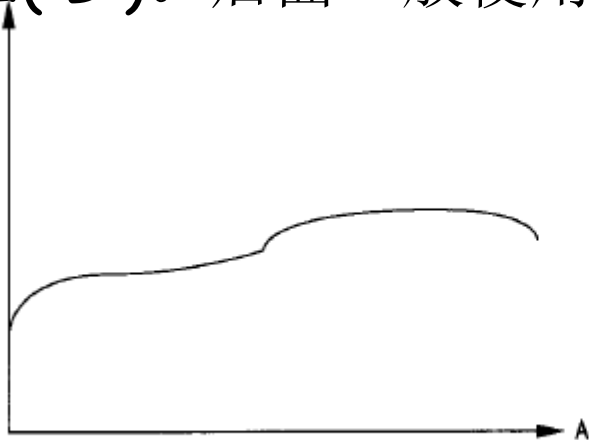
- 鉴于存在以上种种问题，可得出结论：**emp_info**关系模式不是一个好的模式。一个“好”的模式应该不会发生插入异常、删除异常、更新异常，数据冗余应尽可能少。
- 一个关系模式之所以会产生上述问题，是由于存在于模式中的某些**数据依赖**引起的。**规范化理论**正是用来改造关系模式，通过**分解**关系模式来消除其中的某些不合适的数据依赖，以解决插入异常、删除异常、更新异常、数据冗余度大的问题。

6.6 Functional Dependencies

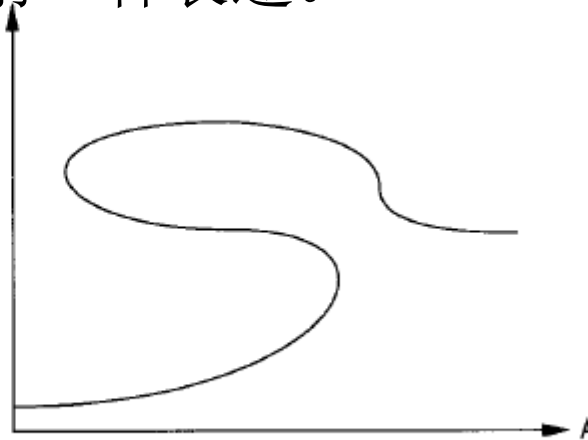


School of software Yunnan University

- **Definition 6.6.1** 给定一个包含至少两个属性 A 和 B 的表 T ，我们说 $A \rightarrow B$ （读做“ A 函数决定 B ”或者“ B 函数依赖于 A ”），当且仅当设计者希望对于表 T 的任何可能的内容（行集合）， T 中的两行不能在 A 上取相同值而在 B 上取不同值。一个更形式化的说法是：给定 T 的两行 $r1$ 和 $r2$ ，如果 $r1(A) = r2(A)$ ，那么 $r1(B) = r2(B)$ 。后面一般使用前一种表述。

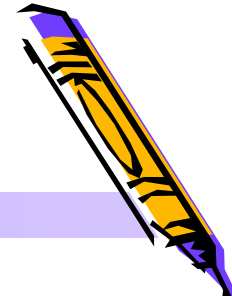


A 函数决定 B ， A 的每个值对应 B 的唯一个值。



A 不函数决定 B ， A 的某些值与 B 的多于一个值相对应。

6.6 Functional Dependencies



■ EXE 6.6.1

$emp_id \rightarrow emp_name$
 $emp_id \rightarrow emp_phone$
 $emp_id \rightarrow dept_name$

emp_info

emp_id	emp_name	...	skill_id	skill_name	skill_date	skill_lvl
09112	Jones	...	44	librarian	93-15-09	12
09112	Jones	...	26	PC-admin		
09112	Jones	...	89	word-proc		
12231	Smith	...	26	PC-admin		
12231	Smith	...	39	bookkeeper		
13597	Brown	...	27	statistics		
14131	Blake	...	26	PC-admin		
14131	Blake	...	89	word-proc	09-15-09	
...	...	...	...	...	...	...

语义：假设一个雇员只登记一个唯一的电话号码，并且多个雇员可以共用一个电话，即
 $emp_phone \nrightarrow emp_id$

图6-16 单个雇员信息表emp_info，满足第一范式

6.6 Functional Dependencies



School of software Yunnan University

- **Definition 6.6.2** 给定表 T 和两个属性集，分别为 $X=A_1A_2...A_k$ 和 $Y= B_1B_2...B_m$ ， X 的一些属性可以与 Y 的一些属性重叠。我们说 $X \rightarrow Y$ （读做“ X 函数决定 Y ”或者“ Y 函数依赖于 X ”），当且仅当设计者的意图是对于表中可能存在的任何行的集合， T 中两行不能在 X 的属性上相同，而同时在 Y 的属性取不同值。注意到两行在 X 的属性上相同是指它们在 X 的所有属性上取相同值，而它们在 Y 的属性上不相同是指在 Y 的任意一个属性上取值不同。更为形式化的表述是，给定 T 的两行 r_1 和 r_2 ，如果 $r_1(A_i) = r_2(A_i)$ 对每一个 X 中的 A_i 成立，那么 $r_1(B_j) = r_2(B_j)$ 对于 Y 中的每一个 B_j 成立。

6.6 Functional Dependencies



School of software Yunnan University

■ Example 6.6.3

emp_id
emp_name
emp_phone
dept_name
dept_phone
dept_mgrname
skill_id
skill_name
skill_date
skill_lvl

} 从一个到大量的技能
可用于一个公司

- (1) emp_id → emp_name emp_phone dept_name
- (2) dept_name → dept_phone dept_mgrname
- (3) skill_id → skill_name
- (4) emp_id skill_id → skill_date skill_lvl

即: emp_id → emp_name
emp_id → emp_phone
emp_id → dept_name

The three FDs of Example 6.6.1 together imply FD(1)

图6-15 没有规范化的雇员信息数据项

6.8 Normal Forms



School of software Yunnan University

□ 第二范式 (2NF)

- Definition 6.8.7 Second Normal Form, a table in which each non-key field is determined by the whole primary key and not part of the primary key by itself.
- 即，若关系模式的 $R \in 1NF$ ，并且每一个非主属性都完全依赖于 R 的键，则 $R \in 2NF$
- Violations of 2NF can only occur on tables that have primary keys formed by more than one field.

6.8 Normal Forms



School of software Yunnan University

□ A Succession of Decompositions to Eliminate Anomalies

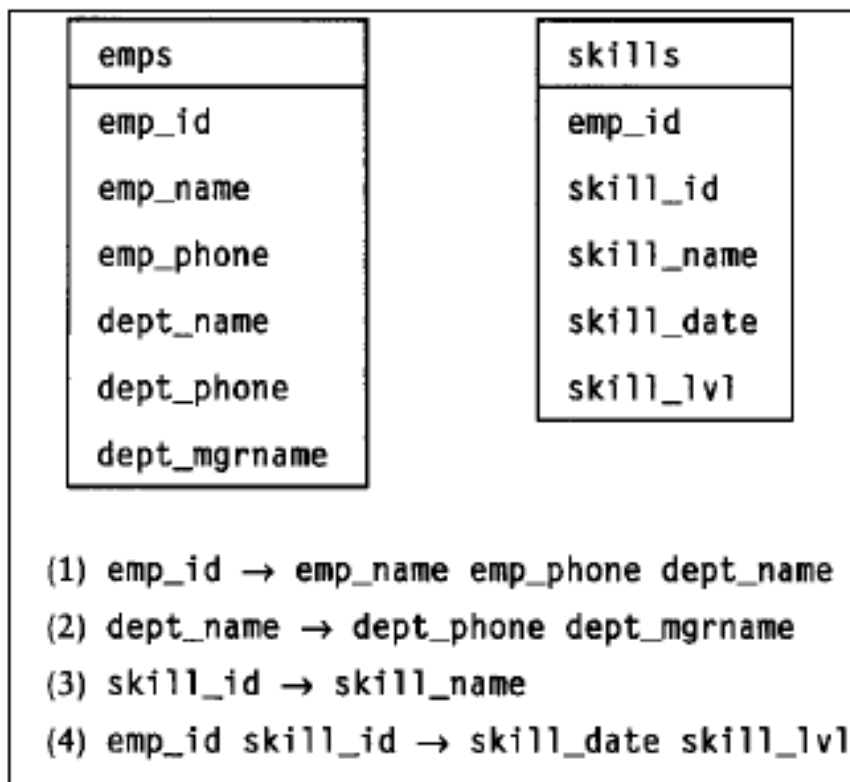


图6-24 具有两个表emps 和skills 的雇员信息模式

6.8 Normal Forms



School of software Yunnan University

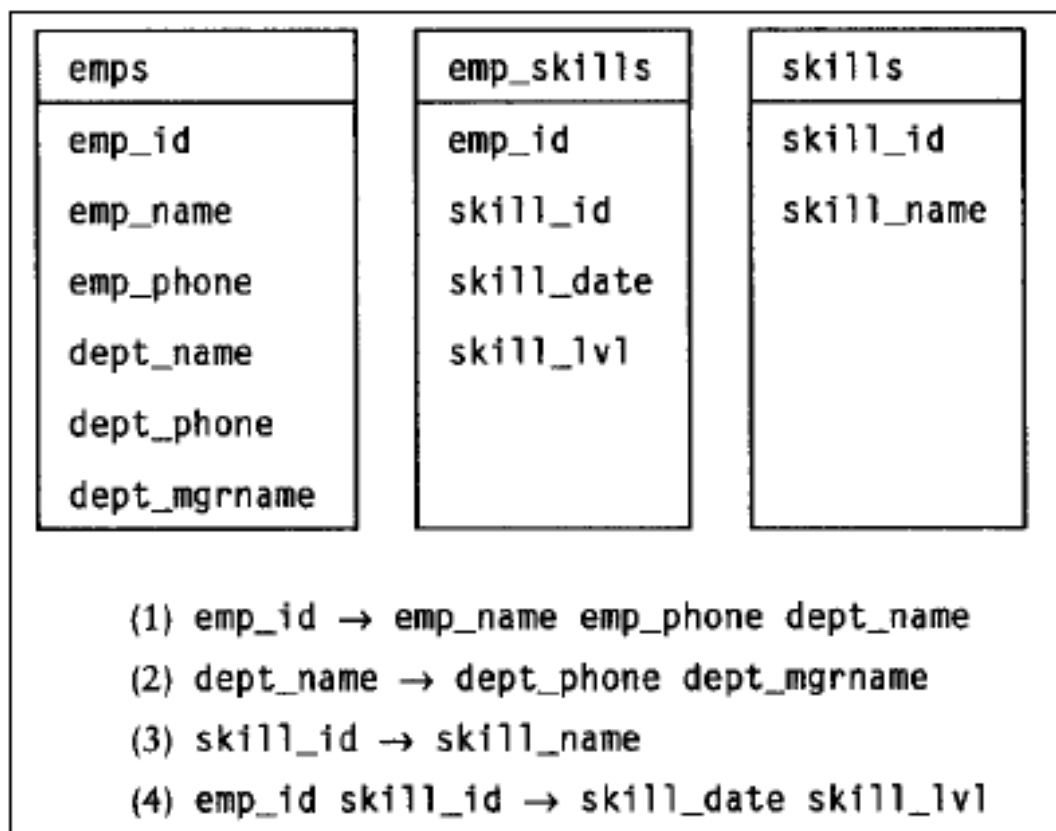


图6-25 具有三个表的雇员信息模式

6.8 Normal Forms



School of software Yunnan University

□ 第三范式 (3NF)

- Definition 6.8.6 Third Normal Form, a table in which none of the non-key fields determine another non-key field.
- Violations of 3NF can occur in any table having more than one non-key field.
- 若关系模式的 $R \in 2NF$ ，并且每一个非主属性既不部分依赖于 R 的键也不传递依赖于 R 的键，则 $R \in 3NF$ 。

6.8 Normal Forms



School of software Yunnan University

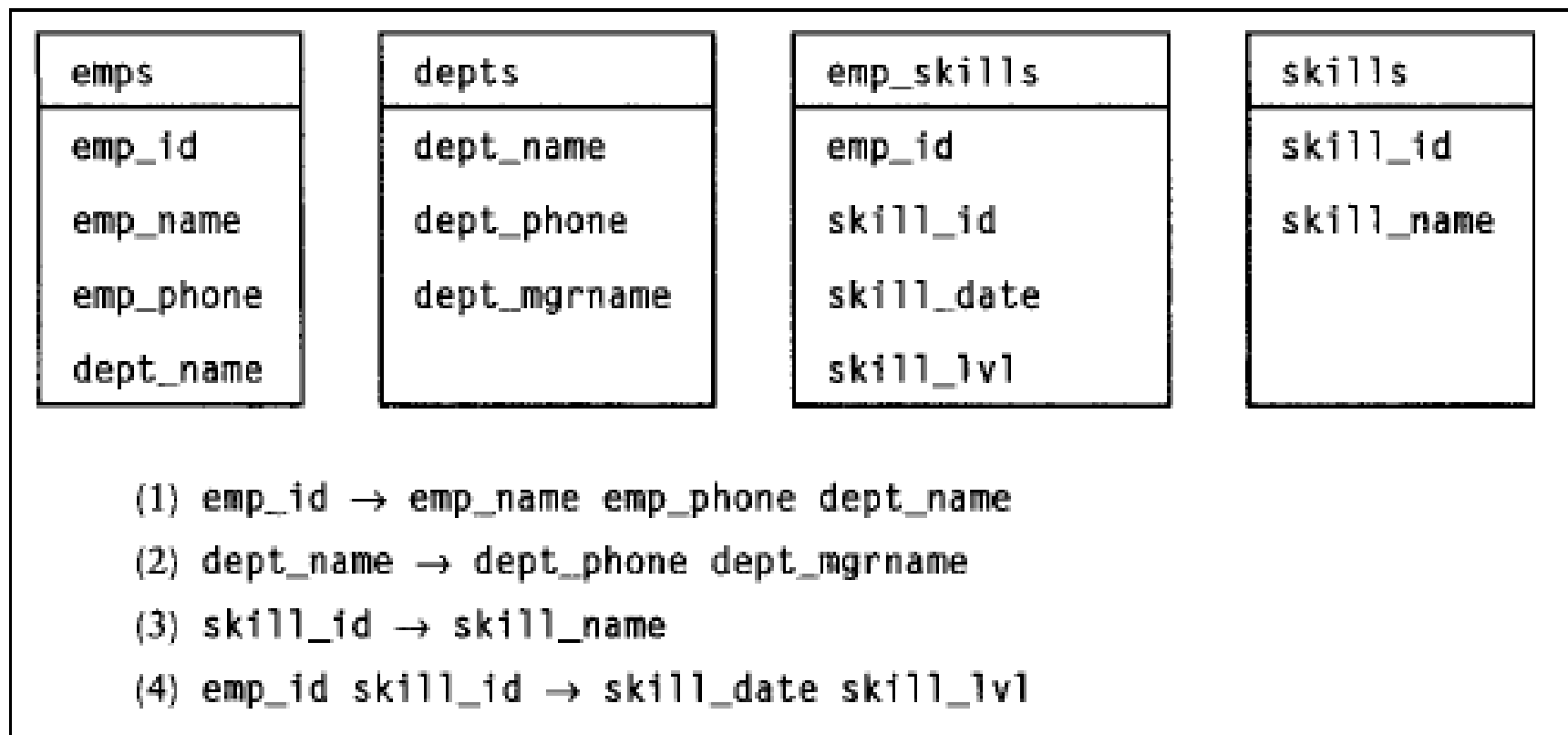


图6-26 符合3NF（也符合BCNF）的雇员信息数据库模式

6.8 Normal Forms



School of software Yunnan University

□ BC范式 (BCNF)

- 给定一个数据库模式和一个通用表 T 以及函数依赖集 F ，令 $\{T_1, T_2, \dots, T_k\}$ 是 T 的一个无损分解，那么对于 F 的一个函数依赖 $X \rightarrow Y$ ，如果对分解中的某一个表 T_i ，有 $X \cup Y \subset \text{Head}(T_i)$ ，则称该函数依赖在 T 的分解中被保持，或者说， T 的分解保持了函数依赖 $X \rightarrow Y$ 。此时，我们也说函数依赖 $X \rightarrow Y$ 在 T_i 中被保持。
- Definition 6.8.3 Every determinant is a key.
- 也就是说， **R 中若每一个决定因素都是键**，则 $R \in \text{BCNF}$ 。

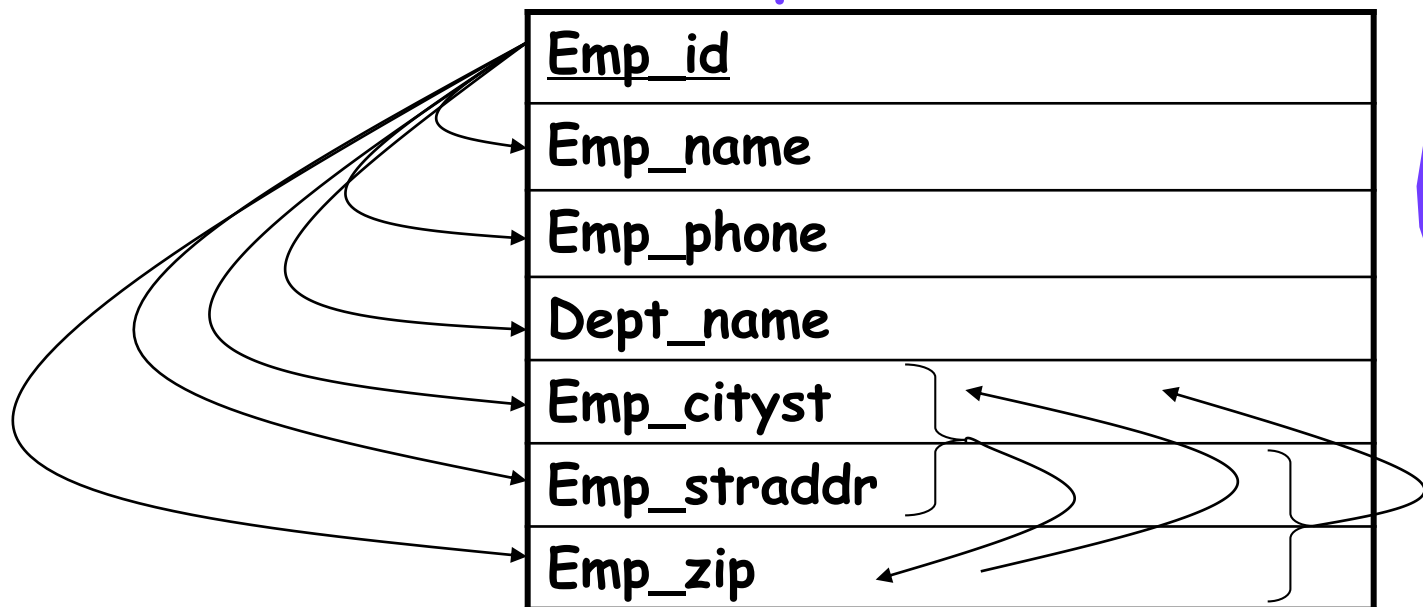
6.8 Normal Forms



School of software Yunnan University

□ BC范式 (BCNF)

Emps



- 满足2NF?
- 满足3NF?

6.8 Normal Forms



School of software Yunnan University

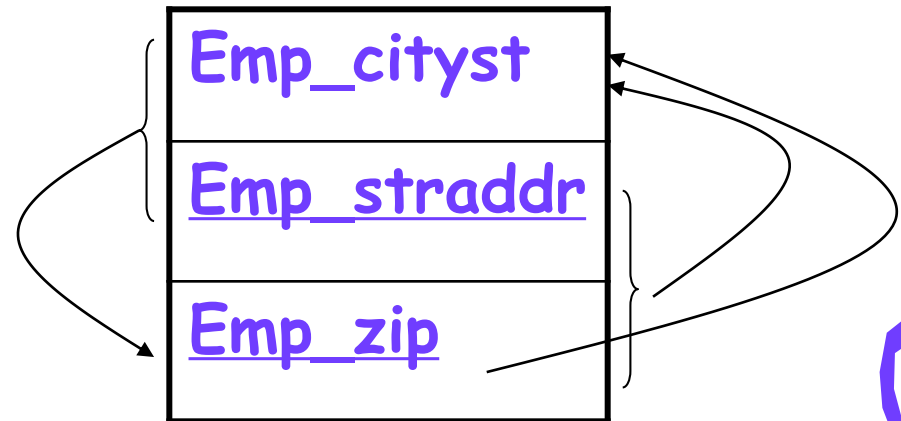
□ BC范式 (BCNF)

■ 3NF decomposition

Emps

<u>Emp_id</u>
Emp_name
Emp_phone
Dept_name
Emp_cityst
Emp_straddr

Empadds



- 满足2NF?
- 满足3NF?
- 满足BCNF?

6.8 Normal Forms



School of software Yunnan University

□ BC范式 (BCNF)

- BCNF violation creates update problem:

Emp_cityst	Emp_straddr	Emp_zip
Kunming	Cuihu	650091
Kunming	Wuhuashan	650091
Kunming	Yangpu	650200

6.8 Normal Forms



School of software Yunnan University

□ BC范式 (BCNF)

■ BCNF decomposition

Emps

<u>Emp_id</u>
Emp_name
Emp_phone
Dept_name
Emp_cityst
Emp_straddr

zipstr

<u>Emp_zip</u>
<u>Emp_straddr</u>

zipcit

<u>Emp_zip</u>
Emp_cityst

- 满足2NF?
- 满足3NF?
- 满足BCNF?

6.8 Normal Forms



School of software Yunnan University

□ BC范式 (BCNF)

- **3NF**和**BCNF**是以函数依赖为基础的关系模式规范化程度的测度。
- 如果一个关系数据库中的所有关系模式都属于**3NF**，则已在很大程度上消除了插入异常和删除异常，如果一个关系数据库中的所有关系模式都属于**BCNF**，那么在函数依赖范畴内，它已实现了模式的彻底分解，达到了最高的规范化程度，消除了插入异常和删除异常。

6.8 Normal Forms



School of software Yunnan University

- 范式是符合某一种级别的关系模式的集合。关系数据库中的关系必须满足一定的要求。满足最低要求的叫第一范式，简称**1NF**，在第一范式基础上进一步满足一些要求的为第二范式，简称**2NF**。其余依此类推，目前有六种范式：**1NF**，**2NF**，**3NF**，**BCNF**，**4NF**，**5NF**。范式之间存在的联系：
- 若 $R(U, F)$ 符合**x**范式的要求，则称**R**为**x**范式，记做： $R \in xNF$

$$5NF \subset 4NF \subset BCNF \subset 3NF \subset 2NF \subset 1NF$$

6.8 Normal Forms



School of software Yunnan University

□ 第一范式 (1NF)

- 例：关系模式SLC (Sno, Sdept, Sloc, Cno, Grade)，语义：Sloc为学生的住处，每个系的学生住在同一个地方。
- (1) 首先分析哪个属性是主码
- $Sno \rightarrow Sdept$, $Sno \rightarrow Sloc$, $Sdept \rightarrow Sloc$ ，但是 $Sno \nrightarrow Cno$, $Sno \nrightarrow Grade$ ，因此Sno不能成为主码
- 主码应该是 (Sno, Cno)
- (2) 分析主码与其他属性的关系：
- $(Sno, Cno) \xrightarrow{f} Grade$, $(Sno, Cno) \xrightarrow{p} Sdept$, $(Sno, Cno) \xrightarrow{p} Sloc$

6.8 Normal Forms



School of software Yunnan University

□ 第二范式 (2NF)

- 把SLC (Sno, Sdept, Sloc, Cno, Grade) 分解为两个关系模式
 - SC (Sno, Cno, Grade)
 - SL (Sno, Sdept, Sloc)
- 其中SC的码是 (Sno, Cno), SL的码是Sno。满足2NF。

6.8 Normal Forms



School of software Yunnan University

□ 第三范式 (3NF)

- SLC (Sno, Sdept, Sloc, Cno, Grade) ——1NF
- SC (Sno, Cno, Grade), SL (Sno, Sdept, Sloc) ——2NF
- $Sdept \rightarrow Sloc$
- 将SL分解为SD (Sno, Sdept) DL (Sdept, Sloc)
- 满足3NF
- SC (Sno, Cno, Grade), SD (Sno, Sdept) DL (Sdept, Sloc) 满足3NF, 也满足BCNF。

6.8 Normal Forms



School of software Yunnan University

□ Definitions of NF:

- 1NF: All fields must contain single values only;
- 2NF: Part of the primary key may not determine a non-key field all by itself.
- 3NF: A non-key field may not determine another non-key fields.
- BCNF: Every determinant is a key.



6.8 Normal Forms



School of software Yunnan University

□ 练习

- 设有关系模式R（运动员编号，比赛项目，成绩，比赛类别，比赛主管），如果规定：每个运动员每参加一个比赛项目，只有一个成绩；每个比赛项目只属于一个比赛类别；每个比赛类别只有一个比赛主管。
 - 请回答下列问题：
 - （1）根据上述规定，写出模式R的基本FD和关键码
 - （2）说明R不是2NF的理由，并把R分解成2NF模式集
 - （3）进而分解成3NF模式集

6.8 Normal Forms



School of software Yunnan University

□ 多值依赖与第四范式（4NF）

- 前面完全是在函数依赖范畴内讨论关系模式的范式问题。如果仅考虑函数依赖这一种数据依赖，属于**BCNF**的关系模式已经很完美了，但如果考虑其他数据依赖，例如多值依赖，属于**BCNF**的关系模式仍然存在问题。
- 例如：设学校中某一门课程有多个教师讲授，他们使用相同的一套参考书。可以用一个关系模式**Teach(C, T, B)**表示课程，教师和参考书之间的关系，如图：

6.8 Normal Forms



School of software Yunnan University

□ 多值依赖与第四范式（4NF）

课程C	教师T	参考书B
数学	邓军	数学分析
数学	邓军	高等代数
数学	邓军	微分方程
数学	陈思	数学分析
数学	陈思	高等代数
数学	陈思	微分方程
物理	邓军	普通物理
物理	邓军	光学物理
物理	王强	普通物理
物理	王强	光学物理

6.8 Normal Forms



School of software Yunnan University

□ 多值依赖与第四范式（4NF）

- 三个属性单独都不能成为候选码，**Teach**具有唯一侯选键（**C**, **T**, **B**），即全码，则这样的结构称为全码结构,因而**Teach** $\in$ **BCNF**。
- 但**Teach**模式中存在一些问题：
- 数据冗余度大：参考书存储多次
- 插入操作复杂：某门课程增加一名任课教师需要增加多行（参考书）
- 删除操作复杂：去掉一本参考书，需删除多行
- 修改操作复杂：修改一本参考书信息，需修改多行
- **BCNF**的关系模式之所以会产生上述问题，原因是参考书的取值和教师的取值是彼此毫无关系的，他们都只取决于课程名，也就是说，关系模式**Teach**中存在一种称为多值依赖的数据依赖。

6.8 Normal Forms



School of software Yunnan University

□ 多值依赖与第四范式 (4NF)

- 定义：（多值依赖, multivalued dependency, MVD）
设 $R(U)$ 是一个属性集合 U 上的一个关系模式， X, Y 和 Z 是 U 的子集，并且 $Z = U - X - Y$ ，多值依赖 $X \twoheadrightarrow Y$ 成立，当且仅当对 R 的任意关系 r ， r 在 (X, Z) 上的每个值对应一组 Y 的值，这组值仅仅决定于 X 值而与 Z 值无关。
- 在Teach关系中，每个 (C, T) 的值对应一组 B 值，而且这种对应与 T 无关。例如（数学，邓军）对应一组值（数学分析，高等代数，微分方程），如果 T 值改变，即（数学，陈思），仍然对应一组值（数学分析，高等代数，微分方程），因此 B 多值依赖于 C ，即 $C \twoheadrightarrow B$

6.8 Normal Forms



School of software Yunnan University

□ 多值依赖与第四范式 (4NF)

- Definition: In an all-key table, part of the key can determine multiple values of at most one other field.
- *4NF applies only to all-key tables.*
- 可以用投影分解法把Teach分解为如下两个4NF关系模式以减少数据冗余:
- CT (C, T) CB (C, B)

6.8 Normal Forms



School of software Yunnan University

□ 多值依赖与第四范式 (4NF)

- 分解后**Teach**关系中的几个问题可以得到解决。
- (1) 参考书只需要在**CB**关系中存储一次。
- (2) 当某一课程增加一名任课教师时, 只需要在**CT**关系中增加一个元组
- (3) 某一门课要去掉一本参考书, 只需要在**CB**关系中删除一个相应的元组。

6.8 Normal Forms



School of software Yunnan University

□ 多值依赖与第四范式（4NF）

- 函数依赖和多值依赖是两种最重要的数据依赖。如果只考虑函数依赖，则属于**BCNF**的关系模式已经很完美了。事实上，数据依赖中除函数依赖和多值依赖之外，还有一种连接依赖，如果消除**4NF**的关系模式中存在的连接依赖，则可以进一步投影分解为**5NF**的关系模式，到目前为止，**5NF**是最终的范式。

6.8 Normal Forms



School of software Yunnan University

□ Definitions of NF:

- 1NF: All fields must contain single values only;
- 2NF: Part of the primary key may not determine a non-key field all by itself.
- 3NF: A non-key field may not determine another non-key fields.
- BCNF: Every determinant is a key.
- 4NF: In an all-key table, part of the key can determine multiple values of at most one other field.

6.6 Functional Dependencies



School of software Yunnan University

□ 对于函数依赖，需要说明以下几点：

- (1) 函数依赖不是指关系模式**R**的某个或某些关系实例满足的约束条件，而是指**R**的所有关系实例均要满足的约束条件。
- (2) 函数依赖和别的数据之间的依赖关系一样，是语义范畴的概念。我们只能根据数据语义来确定函数依赖。
- (3) 数据库设计者可以对现实世界作强制的规定。
- (4) 若 $X \rightarrow Y$ ，则**X**称为这个函数依赖的决定属性集。
- (5) 若 $X \rightarrow Y$ ，并且 $Y \rightarrow X$ ，则记为 $X \leftrightarrow Y$
- (6) 若**Y**不函数依赖于**X**，则记为 $X \nrightarrow Y$

6.6 Functional Dependencies



School of software Yunnan University

□ Logical Implications among Functional Dependencies

- **Theorem 6.6.3 Inclusion Rule.**(包含规则)给定表 T , 其标题是 $Head(T)$ (是一个属性集, 如同2.2节中定义的)。如果 X 和 Y 是 $Head(T)$ 中属性的集合, 并且 $Y \subseteq X$, 那么 $X \rightarrow Y$ 。

6.6 Functional Dependencies



School of software Yunnan University

□ Logical Implications among Functional Dependencies

- 在关系模式 $R(U)$ 中，对于 U 的子集 X 和 Y ，如果 $X \rightarrow Y$ ，但 $Y \not\subseteq X$ 则称 $X \rightarrow Y$ 是非平凡函数依赖。若 $Y \subseteq X$ ，则称 $X \rightarrow Y$ 为平凡函数依赖。
- 对于任一关系模式，平凡函数依赖都是必然成立的，但是它不反映新的语义，对我们分析问题没有什么帮助，所以若不特别声明，我们总是讨论非平凡函数依赖。

6.6 Functional Dependencies



School of software Yunnan University

□ Armstrong's Axioms 阿姆斯特朗公理

■ **Definition 6.6.6 Armstrong's Axioms.** 假设给定一个表 T ，以及 $\text{Head}(T)$ 的子集 X 、 Y 、 Z ，那么我们有下面的蕴涵规则：

- 1) 包含规则 (Inclusion rule) : 如果 $Y \subseteq X$ ，那么 $X \rightarrow Y$ 。
- 2) 传递规则 (Transitivity rule) : 如果 $X \rightarrow Y$ 且 $Y \rightarrow Z$ ，那么 $X \rightarrow Z$ 。
- 3) 增广规则 (Augmentation rule) : 如果 $X \rightarrow Y$ ，那么 $XZ \rightarrow YZ$ 。

■ 像我们在一个函数依赖中列出属性并在它们之间加入空格来表示包含这些属性的集合一样，两个顺序排列的属性的集合表示一个并运算。所以增广规则可以重写为：如果 $X \rightarrow Y$ ，那么 $X \cup Z \rightarrow Y \cup Z$ 。

6.6 Functional Dependencies



School of software Yunnan University

Armstrong's Axioms 阿姆斯特朗公理

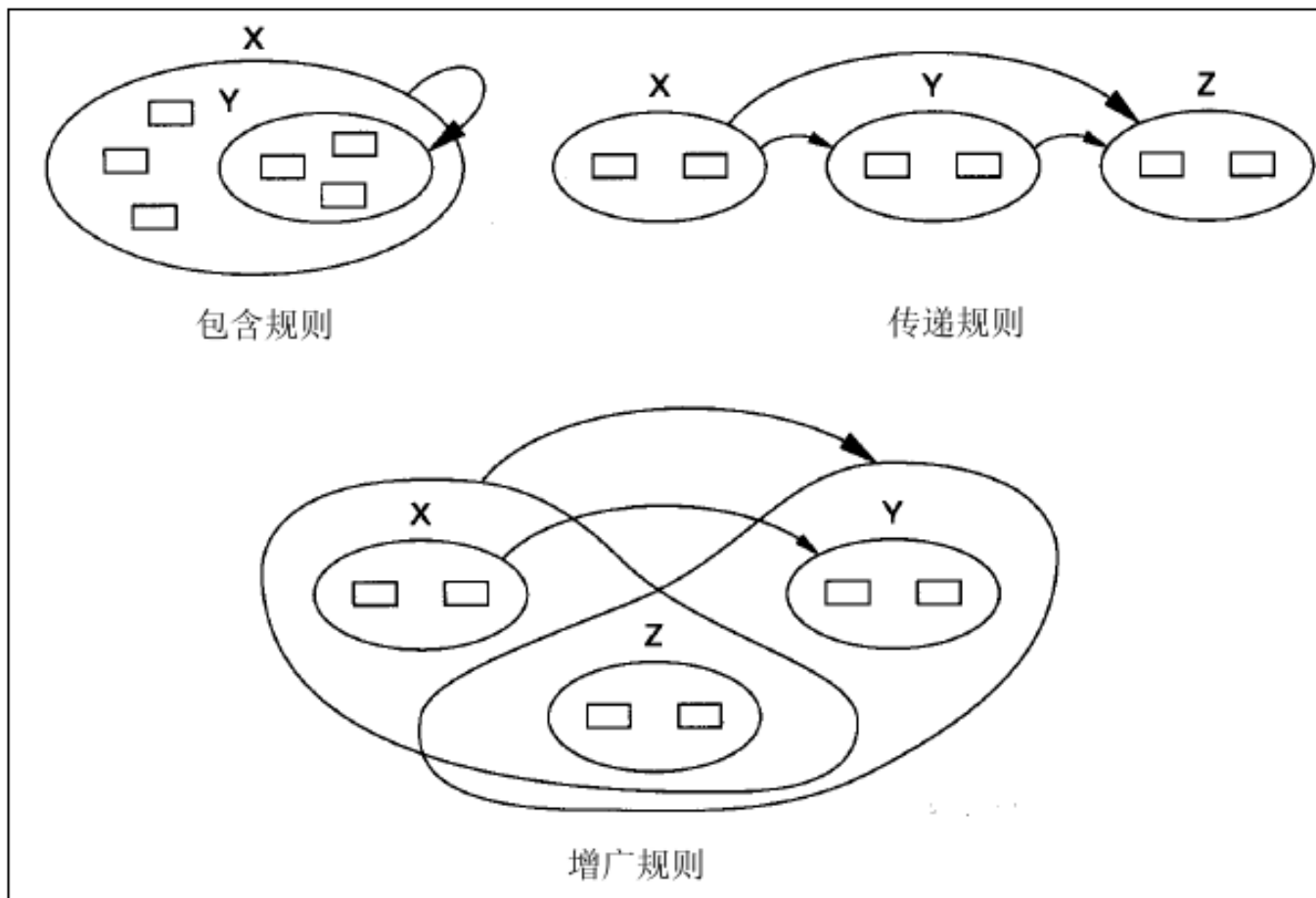


图6-19 阿姆斯特朗公理

6.6 Functional Dependencies



School of software Yunnan University

□ Armstrong's Axioms 阿姆斯特朗公理的一些蕴涵

■ **Theorem 6.6.8 Some Implications of Armstrong's Axioms.** 我们再一次假设下面所有属性集合 W 、 X 、 Y 和 Z 包含在表 T 的标题行中。

- [1] 合并规则 (**Union rule**) : 如果 $X \rightarrow Y$ 且 $X \rightarrow Z$, 那么 $X \rightarrow YZ$.
- [2] 分解规则 (**Decomposition rule**) : 如果 $X \rightarrow YZ$ 那么 $X \rightarrow Y$ 且 $X \rightarrow Z$.
- [3] 伪传递规则 (**Pseudotransitivity rule**) : 如果 $X \rightarrow Y$ 且 $WY \rightarrow Z$, 那么 $XW \rightarrow Z$.
- [4] 集合累积规则 (**Set accumulation rule**) : 如果 $X \rightarrow YZ$ 且 $Z \rightarrow W$, 那么 $X \rightarrow YZW$.

6.6 Functional Dependencies



School of software Yunnan University

□ All valid rules of implication among FDs can be derived from Armstrong's Axioms.

- [Example 6.6.4] List a minimal set of functional dependencies satisfied by the table T.

行#	T			
	A	B	C	D
1	a1	b1	c1	d1
2	a1	b1	c2	d2
3	a2	b1	c1	d3
4	a2	b1	c3	d4

a) 对其他任何属性 $p \in \{A, C, D\}$ 都不会有 $r1(p)=r2(p)$ 而 $r1(B) \neq r2(B)$, 即只要有 $r1(p)=r2(p)$, 则必然有 $r1(B)=r2(B)$

所以:

$A \rightarrow B, C \rightarrow B, D \rightarrow B$

同理:

$B \not\rightarrow A, B \not\rightarrow C$ 且 $B \not\rightarrow D$.

6.6 Functional Dependencies



School of software Yunnan University

□ All valid rules of implication among FDs can be derived from Armstrong's Axioms.

- [Example 6.6.4] List a minimal set of functional dependencies satisfied by the table T.

行#	T			
	A	B	C	D
1	a1	b1	c1	d1
2	a1	b1	c2	d2
3	a2	b1	c1	d3
4	a2	b1	c3	d4

b) $A \rightarrow B$, $C \rightarrow B$, $D \rightarrow B$

所有D值都不同，所以：

$D \rightarrow A$, $D \rightarrow C$

同理，其它属性上都有相同值，所以D不函数依赖于其他任何属性：

$B \not\rightarrow A$, $B \not\rightarrow C$ 且 $B \not\rightarrow D$

且： $A \not\rightarrow D$ 和 $C \not\rightarrow D$

c) $A \not\rightarrow C$ $C \not\rightarrow A$

6.6 Functional Dependencies



School of software Yunnan University

□ All valid rules of implication among FDs can be derived from Armstrong's Axioms.

- [Example 6.6.4] List a minimal set of functional dependencies satisfied by the table T.

行#	T			
	A	B	C	D
1	a1	b1	c1	d1
2	a1	b1	c2	d2
3	a2	b1	c1	d3
4	a2	b1	c3	d4

左边只有一个属性的函数依赖和非函数依赖:

1) $A \rightarrow B, C \rightarrow B, D \rightarrow B$

$D \rightarrow A, D \rightarrow C$

2) $B \nrightarrow A, B \nrightarrow C$ 且 $B \nrightarrow D$.

$A \nrightarrow D$ 和 $C \nrightarrow D$

$A \nrightarrow C, C \nrightarrow A$

We have the following set of FDs

$A \rightarrow B, C \rightarrow B, D \rightarrow ABC$

6.6 Functional Dependencies



School of software Yunnan University

□ All valid rules of implication among FDs can be derived from Armstrong's Axioms.

- [Example 6.6.4] List a minimal set of functional dependencies satisfied by the table T.

行#	T			
	A	B	C	D
1	a1	b1	c1	d1
2	a1	b1	c2	d2
3	a2	b1	c1	d3
4	a2	b1	c3	d4

$A \rightarrow B, C \rightarrow B, D \rightarrow ABC$

d) 任何包含D的属性对都函数决定其他属性，没有左边包含D的新的函数依赖

e) 如果 $P \not\rightarrow Q$ ，就意味着存在行 $r1$ 和 $r2$ 使得 $r1(Q) \neq r2(Q)$ 而 $r1(P) = r2(P)$ 。但是因为 B 在所有行上取相同值，并且我们知道 $r1(BP) = r2(BP)$ 也成立，所以 $BP \rightarrow Q$ 。因此我们不会得到左边包含 B 的新函数依赖。

6.6 Functional Dependencies



School of software Yunnan University

□ All valid rules of implication among FDs can be derived from Armstrong's Axioms.

- [Example 6.6.4] List a minimal set of functional dependencies satisfied by the table T.

行#	T			
	A	B	C	D
1	a1	b1	c1	d1
2	a1	b1	c2	d2
3	a2	b1	c1	d3
4	a2	b1	c3	d4

f) 现在唯一左边不包含 **B** 或者 **D** 的属性对是 **AC**。因为 **AC** 在每一行上有不同的值，所以我们知道 **AC** → **ABCD**

- **AC** → **A** 和 **AC** → **C** 是平凡函数依赖
- **AC** → **B** 可以由 **AC** → **A** 和 **A** → **B** 推导出
- 新的函数依赖只有 **AC** → **D**

A → **B**, **C** → **B**, **D** → **ABC**

6.6 Functional Dependencies



School of software Yunnan University

□ Closure, Cover, and Minimal Cover

- **Definition 6.6.9 Closure of a Set of FDs.**函数依赖集的闭包
给定一个函数依赖集 F ，它作用在表 T 的属性上。定义 F 的闭包为可以从 F 推导出的所有函数依赖的集合，记为 F^+ 。

6.6 Functional Dependencies



School of software Yunnan University

□ Closure, Cover, and Minimal Cover

■ **Definition 6.6.10 FD Set Cover.**函数依赖集的覆盖 表T上的两个函数依赖集合 F 和 G ，如果函数依赖集 G 可以从 F 用蕴涵规则推导出来，或者换句话说，如果 $G \subseteq F^+$ ，那么我们说 **F 覆盖 G** 。如果 F 覆盖 G 且 G 覆盖 F ，那么这两个函数依赖集称为等价的，写作 **$F \equiv G$** 。

■ **Example 6.6.6** consider the two sets of FDs on the set of attributes ABCDE

- $F = \{B \rightarrow CD, AD \rightarrow E, B \rightarrow A\}$
- $G = \{B \rightarrow CDE, B \rightarrow ABC, AD \rightarrow E\}$
- F 覆盖 G

6.6 Functional Dependencies



School of software Yunnan University

□ Closure, Cover, and Minimal Cover

■ Definition 6.6.11 Closure of a Set of Attributes. 属性集的闭包

给定表 T 的函数依赖集 F 和 T 中包含的属性集 X ，我们定义 X 的闭包 X^+ ，作为由 X 函数决定的最大属性集合 Y ，则最大集合 Y 满足 $X \rightarrow Y$ 存在于 F^+ 中。注意，根据包含规则，集合 Y 一定包含了 X 的所有属性，但可以不包含其他任何属性。

6.6 Functional Dependencies



School of software Yunnan University

□ Closure, Cover, and Minimal Cover

■ Algorithm 6.6.12 Set Closure. 集合闭包

```
I = 0; X[0] = X;                                /* integer I, attribute set X[0] */
REPEAT                                           /* loop to find larger X[I] */
    I = I + 1;                                  /* new I */
    X[I] = X[I-1];                             /* initialize new X[I] */
    FOR ALL  $Z \rightarrow W$  in F                /* loop on all FDs  $Z \rightarrow W$  in F */
        IF  $Z \subseteq X[I]$               /* if Z contained in X[I] */
            THEN  $X[I] = X[I] \cup W$ ;    /* add attributes in W to X[I] */
    END FOR                                     /* end loop on FDs */
UNTIL  $X[I] = X[I-1]$ ;                       /* loop until no new attributes */
RETURN  $X^+ = X[I]$ ;                          /* return closure of X */
```

6.6 Functional Dependencies



School of software Yunnan University

□ Closure, Cover, and Minimal Cover

■ [Example 6.6.7]

- F: (1) $B \rightarrow CD$, (2) $AD \rightarrow E$, (3) $B \rightarrow A$

■ Given $X=B$, we determined that $X^+=ABCDE$

■ The order is crucial

- $BCD(1)A(3)E(2)$



6.6 Functional Dependencies



School of software Yunnan University

□ Closure, Cover, and Minimal Cover

- 假使我们需要知道函数依赖 $X \rightarrow A$ 是否可以由函数依赖集 F 推导出来, 我们只需简单地在 F 下使用集合闭包算法计算出 X^+ , 然后看它是否包含 A , 如果包含, 那么 $X \rightarrow A$ 在 F^+ 中, 即它可以由 F 推导出来。
- 表的键恰恰是这个表中可以函数决定所有属性的最小属性集合。为判定 X 是否是一个键, 我们只需要在该表的属性的函数依赖集 F 下计算 X^+ , 看它是否包括了表中全部属性, 然后确认没有 X 的某个子集也满足这一点。

6.6 Functional Dependencies



School of software Yunnan University

□ Closure, Cover, and Minimal Cover

■ Algorithm 6.6.13 Minimal Cover.最小覆盖

这个算法构造最小函数依赖集 M ，它覆盖一个给定的函数依赖集 F 。 M 就是 F 的最小覆盖，或者，在有些文章中，称为 F 的规范覆盖（canonical cover）。

- ✓ Step1.从函数依赖集 F ，创建函数依赖 F 的等价集 H ， H 的函数依赖的右边只有单个属性。

$H = \emptyset;$

FOR ALL $X \rightarrow Y$ in F

FOR ALL A IN Y

$H = H \cup \{X \rightarrow A\};$

END FOR

END FOR

6.6 Functional Dependencies



School of software Yunnan University

□ Closure, Cover, and Minimal Cover

■ Algorithm 6.6.13 Minimal Cover.最小覆盖

- ✓ **Step2.** 从函数依赖集H，顺次去掉在H中非关键的单个函数依赖。一个函数依赖 $X \rightarrow Y$ 在一个函数依赖集H中是非关键的，是指如果 $X \rightarrow Y$ 从H中去掉，得到结果 J，仍然有 $H^+ = J^+$ ，或者 $H \equiv J$ 。就是说，从H中去除这个函数依赖对于 H^+ 没有任何影响。

6.6 Functional Dependencies



School of software Yunnan University

□ Closure, Cover, and Minimal Cover

■ Algorithm 6.6.13 Minimal Cover.最小覆盖

- ✓ Step3. 从函数依赖集 H ，顺次用左边具有更少属性的函数依赖替换原来的函数依赖，只要不会导致 H^+ 改变。
- ✓ Step4. 从剩下的函数依赖集中收集所有左边相同的函数依赖，使用合并规则创建一个等价函数依赖集 M ，它的所有函数依赖的左边是唯一的。

6.6 Functional Dependencies

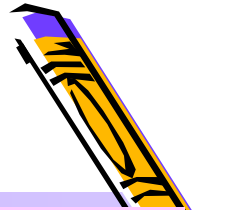


School of software Yunnan University

□ Closure, Cover, and Minimal Cover

- [Example 6.6.8] Construct the minimal cover M for the set F of FDs, which we number and list below.
 - $F: (1)ABD \rightarrow AC, (2)C \rightarrow BE, (3)AD \rightarrow BF, (4)B \rightarrow E$
- Step1. 利用分解规则找到 F 的等价函数依赖集 H , 其右边都是单属性
 - $H=(1)ABD \rightarrow A, (2)ABD \rightarrow C, (3)C \rightarrow B, (4) C \rightarrow E, (5)AD \rightarrow B, (6)AD \rightarrow F, (7)B \rightarrow E$
- Step2. 去除 H 中的非关键函数依赖
 - (1) $ABD \rightarrow A$ is trivial, so it can be removed.

6.6 Functional Dependencies



School of software Yunnan University

□ Closure, Cover, and Minimal Cover

■ [Example 6.6.8] Construct the minimal cover

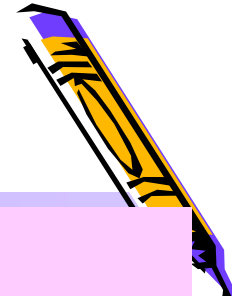
$H = (1) ABD \rightarrow A,$
 $(2) ABD \rightarrow C, (3) C \rightarrow B,$
 $(4) C \rightarrow E, (5) AD \rightarrow B,$
 $(6) AD \rightarrow F, (7) B \rightarrow E$

■ Step2. 去除H中的非关键函数依赖

- (1) $ABD \rightarrow A$ is trivial, so it can be removed.
- (2) $ABD \rightarrow C$ can not be derived from the other FDs in H
- (3) $C \rightarrow B$ is essential and remains in H.
- (4) $C \rightarrow E$ is inessential
- (5) $AD \rightarrow B$ is essential and remains in H
- (6) $AD \rightarrow F$ is essential and remains in H
- (7) $B \rightarrow E$ is essential and remains in H

$H = (2) ABD \rightarrow C,$
 $(3) C \rightarrow B, (5) AD \rightarrow B,$
 $(6) AD \rightarrow F, (7) B \rightarrow E$

6.6 Functional Dependencies



School of software Yunnan University

□ Closure, Cover, and Minimal Cover

■ [Example 6.6.8] Construct the minimal cover

$H = (1) ABD \rightarrow A, (2) ABD \rightarrow C, (3) C \rightarrow B, (4) C \rightarrow E, (5) AD \rightarrow B, (6) AD \rightarrow F, (7) B \rightarrow E$

■ Step3. 简化H, 简化左边属性

- $ABD \rightarrow C$ 能否替换为 $BD \rightarrow C$, 看 $BD + (H下)$ 与 $BD + (J下)$ 是否等价, 它们分别为 BDE 和 $BDCE$ 所以不等价
- $H = (1) AD \rightarrow C, (2) C \rightarrow B, (3) AD \rightarrow F, (4) B \rightarrow E$

■ Step4. 利用合并规则

- $H = (1) AD \rightarrow CF, (2) C \rightarrow B, (3) B \rightarrow E$

$H = (2) ABD \rightarrow C, (3) C \rightarrow B, (5) AD \rightarrow B, (6) AD \rightarrow F, (7) B \rightarrow E$

6.7 Lossless Decompositions



- Def 有损分解是指，表T被分解后，不能通过将分解出的表连接起来而恢复原始表的所有信息。这不是因为我们没有得到所有以前存在的行，而是因为得到了原先没有的行。
- Definition. 对于任何表T以及函数依赖集F，T的一个分解是一个表的集合 $\{T_1, T_2, \dots, T_k\}$ ，它具有以下性质：（1）对于 $i = 1, 2, \dots, k$ ， $Head(T_i)$ 是 $Head(T)$ 的一个子集；（2） $Head(T) \subseteq Head(T_1) \cup Head(T_2) \cup \dots \cup Head(T_k)$ ；（3）对于T的任何内容，F中的函数依赖都保证 $T = T_1 \bowtie T_2 \bowtie \dots \bowtie T_k$ ，则称表T的分解 $(T_1, T_2, \dots, T_k)$ 相对于函数依赖集F是无损分解，有时候也称为无损连接分解。

6.7 Lossless Decompositions



School of software Yunnan University

■ Example 6.7.1 A Lossy Decomposition

The reason we lost information in the decomposition of Example 6.7.1 is that the attribute B has duplicate value on distinct rows of the factored tables.

ABC		
A	B	C
a1	100	c1
a2	200	c2
a3	300	c3
a4	200	c4

JOIN BC		
	B	C
a1	100	c1
a2	200	c2
a2	200	c4
a3	300	c3
a4	200	c2
a4	200	c4

6.7 Lossless Decompositions



School of software Yunnan University

■ Example 6.7.3 Table ABC with a Functional Dependency

B → C

ABC		
A	B	C
a1	100	c1
a2	200	c2
a3	300	c3
a4	200	c2

AB	
A	B
a1	100
a2	200
a3	300
a4	200

BC	
B	C
100	c1
200	c2
300	c3

This means attribute B is a **key** for table BC.

Although duplicate values for column B can occur in table AB, every row in AB joins with a **unique** row in table BC.